

A Project Report on
CREDIT CARD FRAUD DETECTION USING ML

Submitted in the Partial Fulfilment of the Requirements for the Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

by

Aakash Kumar (200080101007)

Sudhir Yadav (200080101084)

Sobhit Kumar (200080101083)

Shafiq Rasool (200080101076)

Under the supervision of
Mukesh Rajput
Assistant Professor



Submitted to the
Department of Computer Science and Engineering
School of Engineering & Engineering (SoEC)
Dev Bhoomi Institute Of Technology(DBIT)
Veer Madho Singh Bhandari Uttarakhand Technical University,
Uttarakhand-248007

TABLE OF CONTENTS

			Page No.
Candidate's Declaration			i
Certificate			ii
Abstract			iii
Acknowledgements			iv
Contents			
List Of Figures			
Chapter	Description		
1		INTRODUCTION:	
	1.1	Description	1
	1.2	Problem Formulation	11
	1.3	Proposed Solution	11
	1.4	Scope of the project	13
2		LITERATURE SURVEY	14
3		SYSTEM DESIGN	
	3.1	Functional Requirements	15
	3.2	Non functional Requirements	15
	3.3	Specific Requirements	15
	3.4	Use-Case Diagram and description	16
4		IMPLEMENTATION DETAILS	
	4.1	Algorithms/ Methods used	18
	4.2	Working of project	20
5		RESULT ANALYSIS	44
6		FUTURE SCOPE & CONCLUSION	46
REFERENCES			48

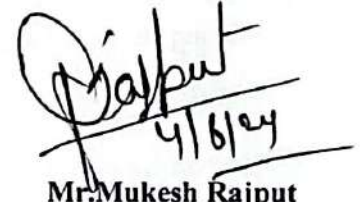
LIST OF FIGURES

Figure No.	Figure Caption	Page No.
1	System Architecture	12
2	Activity Diagram	16
3	Flow Diagram	16
4	Use-Case Diagram	17
5	Class Diagram	17
6	Logistic Curve	19
7	SVM Model Graph	21
8	Decision Tree	22
9	Distribution of Transaction Amount and Time	24
10	Learning Curves of Models	44

CERTIFICATE

This is to certify that the project report entitled "**Credit Card Fraud Detection Using ML**" being submitted by **Akash kumar, Sudhir yadav, Shafiq Rasool Wani and Sobhit kumar** in partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science and Engineering to the DBIT (Dev Bhoomi Institute of Technology) is a record of bonafide work carried out by them under my guidance and supervision.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any Degree

A handwritten signature in black ink, appearing to read 'Mukesh Rajput', with the date '4/6/24' written below it.

Mr. Mukesh Rajput
Assistant Professor
Department of Computer Science and Engg. Dev
Bhoomi Institute of Technology, Dehradun
(Uttarakhand) INDIA

ABSTRACT

With the extensive use of credit cards, fraud appears as a major issue in the credit card business. It is hard to have some figures on the impact of fraud, since companies and banks do not like to disclose the amount of losses due to frauds. At the same time, public data are scarcely available for confidentiality issues, leaving unanswered many questions about what is the best strategy. Credit card fraud avoidance has been the most popular problem in the de

veloped world. In this case, credit card fraud is identified by fraudulent transactions. Since e-commerce sites are becoming more popular, credit card fraud is becoming more common. When a credit card is stolen it is used for dishonest reasons, a fraudster uses the credit card information for his own purposes, and it is called credit card theft. In order to track online fraud transactions, the new technology employs a variety of methods. To increase the consistency of the proposed scheme, we used a Machine learning algorithm to find suspicious transactions. It is built on a supervised learning algorithm. After the dataset has been categorised, analysed and visualised then a confusion matrix is established. Two most important algorithms such as Logistic regression and random forest have been applied for the purpose.

ACKNOWLEDGEMENT

At this ecstatic time of presenting this dissertation, first, the author bows to almighty God for blessing with enough patience and strength to go through this challenging phase of life.

I would like to express a deep sense of gratitude and thanks to those people who have helped me in the accomplishment of this B. Tech. thesis.

First and foremost, I would like to thank my supervisor, **Mr. Mukesh Rajput** for their expertise, guidance, enthusiasm, and patience. These were invaluable contributors whose insightful guidance helped to the successful completion of this dissertation and spent many hours patiently answering questions and troubleshooting the problems.

Finally, I would like to thank all faculty, college management, administrative and technical staff of **School of Engineering & Computing, Uttarakhand Technical University, Dehradun** for their encouragement, assistance, and friendship throughout my candidature.

Date: 4/06/2024

Aakash Kumar

Sobhit Kumar

Shafiq Rasool

Sudhir Yadav

1. Introduction

1.1 Description

Financial fraud is a growing concern with far reaching consequences in the government, corporate organizations, finance industry. In today's world, high dependency on internet technology has enjoyed increased credit card transactions, but, credit card fraud has also accelerated as online and offline transaction. As credit card transactions become a widespread mode of payment, focus has been given to recent computational methodologies to handle the credit card fraud problem. There are many fraud detection solutions and software which prevent frauds in businesses such as credit card, retail, e-commerce, insurance, and industries.

Data mining technique is one notable and popular methods used in solving credit fraud detection problem. It is impossible to be sheer certain about the true intention and rightfulness behind an application or transaction. In reality, to seek out possible evidences of fraud from the available data, using mathematical algorithms is the best effective option. Fraud detection in credit card is truly the process of identifying those transactions that are fraudulent into two classes of legit class and fraud class transactions. Several techniques are designed and implemented to solve credit card fraud detection.

Credit card transaction datasets are rarely available, highly imbalanced and skewed. Optimal feature (variables) selection for the models, suitable metric is most important part of data mining to evaluate performance of techniques on skewed credit card fraud data. A number of challenges are associated with credit card detection, namely fraudulent behaviour profile is dynamic, that is fraudulent transactions tend to look like legitimate ones. Credit card fraud detection performance is greatly affected by type of sampling approach used, selection of variables and detection technique used. In the end, conclusions about results of classifier evaluative testing are made and collated.

Credit cards are now the most preferred way for customers to transact either offline or online. There are a number of reasons, as illustrated below, due to which consumers are slowly shifting from debit card transactions to credit cards, especially in developing countries like India.

- Lucrative cashback and reward point options are present for each credit card transaction. These are generally not offered by financial institutions for debit cards.
- Tie up credit cards with online and offline merchants, especially during festive seasons like Diwali, Eid, and Christmas, to offer further discounts on transactions. Several online merchants run their own promotional campaigns, which are tied up with credit cards—for example, Amazon Prime day, which happens at least once a year.
- Immediate needs can be fulfilled (for example, medical emergencies, lifetime events, etc.) quickly instead of having sufficient account balance for the same. Most credit cards offer 0% EMI options, so it makes it all the more worth pursuing this goal.

- Having a good credit history helps to build a good CIBIL score which then, in turn, helps customers to avail themselves better and competitive interest rates on longer-term needs like home loans or car loans.
- Credit cards are tailored to suit individual customer needs. For example, customers who want to use a credit card for daily usage are usually offered a card with no annual fees or joining fees (marketed as lifetime free credit cards). On the other hand, we have premium cards with annual fees or joining fees for affluent people who offer golf membership, airport lounge access, seamless transactions at international and domestic merchants with lower transaction transfer fees, 5x to 10x reward points, etc.

However, with all these advantages, we also have the additional advantage of the ease of usage without having to carry currency around, and we can get a record of all our digital transactions through credit card statements far more easily compared with cash transactions or bank statements. One downside that has been witnessed over the past few years of this increasing digital phenomenon is the rise of fraud on the credit card. Fraud can be of several types, as we will try to understand a bit later on in this blog.

Before going into details of credit card fraud detection, let us try to understand the size of the overall credit card industry, especially in the large western economies like in some of the European countries, the US, and the UK. Below are some of the numbers around the credit card industry in general at a worldwide level.

- There are 1.06 billion credit cards in use in America and 2.8 billion credit cards worldwide.
- A US citizen, on average, has four active credit cards.
- In the European Union (EU), the number of cards carried per person ranges from 0.8 to 3.9
- In the UK, there were 32.3 million people with credit cards or charge cards in 2016, which roughly translates to 6 in every ten adults. The numbers have only grown since then from 2016 to now.
- There were 368.92 billion card transactions worldwide in 2018. However, the average value per card payment is decreasing in most of the major economies, as a credit card is used more and more as a preferred financial product compared to other means. The average value per card payment drop indicates that customers are using a credit card more and more for daily use compared to one-off events like big purchases.

Impact of COVID-19

The impact of the Covid-19 pandemic worldwide has now given more impetus to digital transactions compared to cash transactions, especially as customers are more hygiene aware now

- Cash volume in the UK for financial transactions has dropped by about 60% as of 2020
- In the US, about 28% population has stopped using cash altogether

- Online shopping increased from 19% to 28% in the UK from 2019 to 2020. It is expected to go up further in 2021

The numbers above indicate that credit card usage and popularity have gained significant ground due to the pandemic. Contactless payment has also gained a lot of impetus, with few countries making it mandatory. However, on the flip side, travel and work-related expenses have dropped as a large part of the working population (which also coincidentally is a large part of the customer base for credit cards) is working from home instead of the office. Also, international travel has taken a severe hit due to fears related to pandemic spread.

In other words, the pandemic has really accelerated the digitization of payments. In contrast to previous years, in 2020, it was not the technology around financial transactions that changed, but the way the consumers used it. Most consumers who had never used digital transactions before were forced to learn and use them during the pandemic. However, with this, consumer anxiety about fraud has also increased manifold.

Credit Card Fraud

There is an explosion of demand for new payment methods. With new payment methods, we have an extremely complex backend which makes fraud detection all the more hard. We have nearly 1.8 billion Euros on average of fraudulent transactions detected in Europe every year.

Global fraud has increased by almost three times, from \$9.84 billion to \$32.39 billion in less than a decade (2011 to 2020). Fraud can be broadly categorized into the following types:

- **Card Not Present (CNP) fraud:** These are mostly digital frauds for which the physical card need not be present at the point of transaction (POS). This usually means online payments. It is also known as "*remote purchase*" fraud.
- **Card Present (CP) fraud:** This is as expected and, as the name suggests, the opposite of the above, for which the physical card needs to be present at the POS site.
- **Mail and telephone order (MOTO) fraud:** Instances of stolen card details being used over the phone or mail (more popular in western countries like the US compared to India)
- **First-party fraud:** In this case, the customer itself is the fraudster. He or she might have run into a financial crisis like loss of job, medical emergency, or sometimes actual malicious intention of not paying back the credit card bank their dues.
- **Identity fraud:** In this case, the customer is the victim, and the fraudster is someone else. For example, says Shyam, the fraudster knows that Ram, the victim, has a very good credit score and other credentials. Shyam applies for the credit card in Ram's name, and the bank, thinking that it is actually Ram who has applied, approves the credit card (this is more likely to happen in the US and other western countries as customers can apply for a credit card online without providing any documents, only a few personal information is required. In India, it is less likely as stringent screening processes like in-person verification, and salary verification, etc., are carried out). Shyam then intercepts the card through either change of

address by setting up an online profile and starts transacting on the card. At the end of the month, Ram receives the statement in his email and refutes the charges, and this is when the bank knows that identity fraud has happened. This can occur either at the start of the credit card lifecycle, in which case it is known as a fraudulent application, or anytime in between the lifecycle, in which case it is known as account takeover.

- **Plastic fraud:** In this scenario, it is generally one-off transactions or a few transactions that are fraudulent on the credit card instead of all the transactions. A popular example can be if Ram loses his Wifi enabled credit card at a shopping mall while having his lunch. Shyam happens to spot the credit card and, instead of volunteering to return to the rightful owner, goes on a shopping spree in the mall doing "tap and pay" or "contactless" transactions instead of providing pin (As per the latest VISA regulation in India, we can transact upto Rs 5,000 using "tap and pay" or "contactless" without giving card pin). Ram, unfortunately, spots the transaction alerts later on his registered mobile or email and immediately blocks the card, but by then, the fraudster has done the damage. This is an example of a lost or stolen card scenario. Similarly, a counterfeit (CFT) credit card can also be created even though with current EMV chip technology, this has become tougher (it was much easier in older credit cards that did not have EMV chips and had only magnetic stripes). CNP, CP, CFT, and MOTO types of frauds belong under the larger umbrella of plastic frauds.

Credit Card Fraud in the United States

The United States has its own banking and finance system, which is different from the rest of the world. The United States is the world's #1 in terms of the size of the economy, and it is observed that Americans have a particular affinity towards credit cards, or we can say towards credit in general. As a result of this, that country is a large target for external hackers, and credit card fraud is more likely to happen in the US than in any other part of the world.

- The US reports the largest credit card losses in the world, which is close to 38.6% of the whole world. Credit card fraud is the most common form of fraud that occurs in the United States.
- Credit card fraud has been on the rise year after year for the last five years. At the same time, total fraud and identity-based frauds have decreased.
- CNP fraud is 81% more likely to happen in the US compared to CP fraud.
- CNP fraud hit \$4.57 billion in the US in 2016, rising about 34% year on year.
- Georgia, Nevada, Florida, and California are the states whose resident population is highly susceptible to fraud. Florida and California also have larger avenues to spend related to travel and entertainment like Casinos and sea sports.
- About 80% of the credit cards in the US have been compromised at some point in time or other.
- About three-quarters of Americans (~73%) are concerned that their financial account, email, or social profiles can be hacked.

Credit Card Fraud stats in the European Union (EU)

The European Union includes all European countries (except the UK after the infamous Brexit), the Nordics, and several other key countries like Switzerland, Monaco, and Liechtenstein. These countries have their own specific rules, especially where the common currency is not Euro. We will use a financial terminology known as bps instead of percentage (For example, 0.01% translates to 1 bps, you might have heard of this when RBI cuts repo rates, etc.)

- The fraud value for cards issued within Europe is estimated to be 1.8 billion Euro in 2016. (as per European central bank)
- 73% of the above comes from CNP payments, 19% from transactions at POS (point of sale) terminals, and 8% from ATM (automated teller machines). It is worthwhile to note that CNP fraud has increased while card-present fraud has decreased. This suggests a migration from physical fraud to digital or online fraud, which is expected as financial institutions make digital access to credit cards easy for customer convenience. (as per European central bank)
- Portugal is the only country which is an exception which has more point of sale or POS fraud than card not present or CNP fraud (as per European central bank)
- The fraud level as a portion of the transaction value ranges from 0.5 bps (basis points) for cards issued in Poland to 7.3 bps for cards issued in the Netherlands in terms of value, and from 0.2 bps in Poland to 4.3 bps for credit cards issued in France in terms of volume. (as per European central bank)
- In general, countries with voluminous card markets (high volume of transactions and greater value per transaction like the UK and France) also experience a high level of card fraud (as per the European central bank)
- The Netherlands has 0.6 bps fraud, Denmark 1.3 bps, Norway 1.6 bps has the lowest ratio of fraud versus legitimate purchases, compared with 53 bps in France and 50 bps in the UK.

As per one of the reports, the Netherlands and Nordic countries are excellent examples of fraud control best practices in Europe due to well-managed fraud and risk prevention services thanks to their pan-European processors, which cover fraud prevention expertise across multiple country borders. In absolute contrast, the UK and France continue to experience higher card fraud losses, mainly from CNP fraud on internet purchases, lost or stolen card fraud, or fraud losses on domestic cards used across multiple country borders.

- The share of fraud is much higher for credit cards versus debit cards, showing that fraudsters prefer to do credit card fraud than debit cards.
- Fraud share can further be broken down into region-wise components, which helps us observe that the majority of fraud is within the European Union itself.
 - 43% of fraud within the European Union outside the domestic country.
 - 35 % of fraud is within the domestic country of the card issuer.
 - 22% of fraud is outside the domestic country and European Union.

Credit fraud in France

France has its own system for cards known as CB (cartes bancaires). This means it has some different rules governing payments, on top of the standard rules for European countries.

It is very much possible and feasible that these rules make committing credit card fraud within France (that is, domestic fraud) far more difficult. Compared with the UK, the fraud rates in France are still lower and sizable.

- Domestic fraud losses on French cards have stabilized at about 3.2 bps.
- On the other hand, the fraud rate on French cards used abroad outside European countries is 16 times higher than on domestic transactions in 2017, while for foreign cards used in France, the rate was 12.1 times higher.
- Identity theft of card details accounted for 66.1% of total domestic card fraud losses in France.
- The main methods of compromise responsible for fraud losses are as below:
 - Lost and stolen fraud (16.3%)
 - CNP fraud (72.3%) based on theft of card credentials

Together, the two categories accounted for 88.6% of losses as of 2017.

Credit Fraud in the Netherlands

The Netherlands brought in a new digital ID service (iDIN) in 2016. This collaboration between Dutch banks serves to increase online security, especially for domestic card usage.

Dutch shoppers rely far more on debit cards than credit cards, and the country also has a popular transfer service where customers can pay online from their bank account.

Overall, levels of credit card fraud in the Netherlands are low and have decreased substantially in recent years.

- Fraud levels had reduced significantly from €33.3 million in 2013 to €12.6 million in 2018
- 39% of card fraud losses in 2018 occurred on debit cards. This is down from 57% in 2017
- While debit card fraud has fallen substantially, internet banking fraud actually increased in 2018. This shows a similar trend in other western countries as well, where the fraudster prefers digital fraud over physical fraud. This was usually a result of phishing techniques, including "scam emails and text messages, fake apps, fake invoices, identity fraud, and deception of financial employees of companies (known as CEO fraud)."

This may also be somewhat thanks to the popularity of IDEAL, the bank transfer system mentioned above. Fewer customers use credit cards in general, and bank accounts themselves may be a juicier

target for fraudsters since the fraudster has direct access to a known amount that can be cashed immediately.

Regardless compared to other European countries, the Netherlands can be considered a success story in tackling credit card fraud to a large extent.

Credit Card Fraud in Denmark

As explained above, we have somewhat conflicting data about Danish rates of fraud (as a percentage of total card payment value). The European Central Bank (ECB) assigns Denmark the highest ratio of fraud to total payments. Meanwhile, other important sources like Nets.eu states that it has one of the lowest ratios.

Part of the cause for this may simply be related to the timing of the reports as the ECB report was published in 2016, while Nets.eu published its report later.

- CNP losses rose significantly from 2014-16 and *"show no signs of slowing as fraudulent attacks continue to migrate across Europe, away from France and the UK."*
- Denmark also has an abnormally high level of lost and stolen fraud (52.7% of total losses), perhaps due to high credit limits.
- In Q2 2018, contactless card fraud made up 65% of all fraudulent card payments, despite only 56% of all payments being contactless. In other words, contactless represents a disproportionate amount of card fraud in Denmark.

What makes credit card fraud detection hard?

After understanding the gravity of the fraud situation worldwide, particularly in the United States and some of the major European countries, the next automatic question that comes to mind is how we prevent this fraud and the damage it causes to the overall economy and especially to the customer sentiments and trust in financial institutions. Also, let us discuss some of the challenges that we face while dealing with credit card fraud as below.

- **Data imbalance:** The fraud and non fraud data are generally much skewed. To give an example in the sample open-source dataset that we will be dealing with here, we have 492 frauds out of a total of 2,84,807 transactions. This is roughly only 0.172% of all the transactions. So it is easy to achieve almost 99% accuracy with a naive model which just predicts all the transactions as non fraud.
- **Customer friction:** The most likely outcome if a model predicts a current transaction as fraud is to decline the transaction outright to prevent any financial loss. However, we will soon see that it sometimes proves to be a bone of contention with genuine customers, who might get declined if the model has too many false positives or Type 1 errors. Though we

may never achieve 100% accuracy in a real-world scenario, it is desirable for the model to be as accurate as possible to minimize any real customer friction.

- **Real-Time Detection:** For most of the fraud detection models in practice they have to work under very stringent timing conditions. We can take an example of a transaction-level fraud detection model. This model has to run and give the decision as to whether the current transaction is fraud or not within a fraction of a second. If we employ a time-consuming but highly accurate model, we might irritate the customer who is waiting to do the transaction, and if we process too fast, we may improve on customer experience, but it might lose out on accuracy. So it is a very thin line that we have to tread on while developing such fraud detection models.

Let us now discuss the dataset that we will be working with to build models and decide their effectiveness in fraud detection.

Dataset description

The dataset contains transactions made by credit cards in September 2013 by European cardholders.

This dataset presents transactions that occurred in two days, where we have 492 frauds out of 284,807 transactions. The dataset is highly unbalanced. The positive class (fraud transactions) accounts for 0.172% of all transactions.

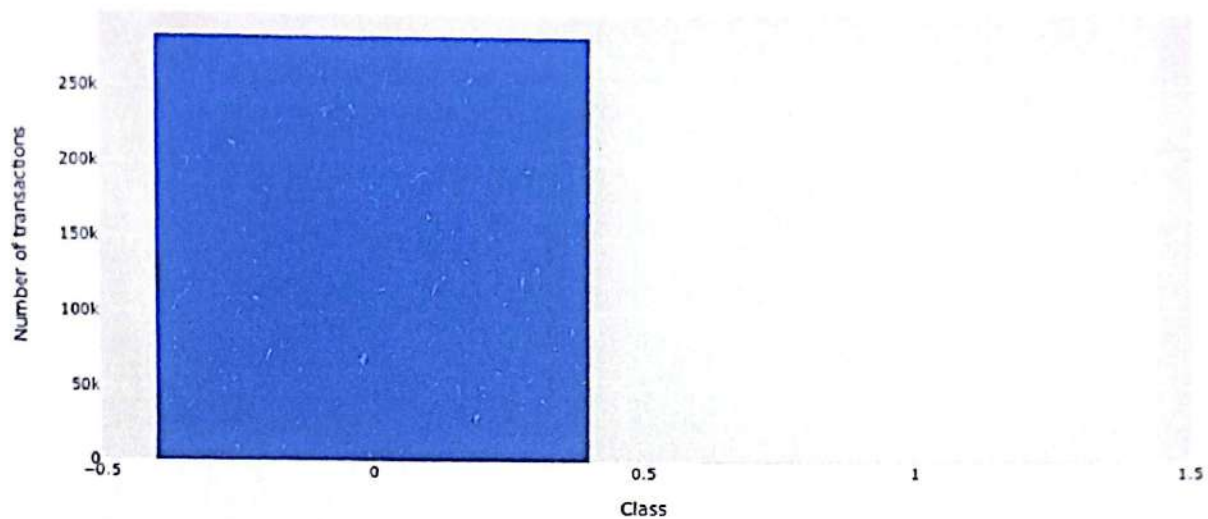
It contains only numeric input variables, which are the result of a PCA transformation. Unfortunately, due to confidentiality issues, the original features and more background information about the data are not provided. Features V1, V2, ... V28 are the principal components obtained with PCA, the only features which have not been transformed with PCA are 'Time' and 'Amount.' Feature 'Time' contains the seconds elapsed between each transaction and the first transaction in the dataset. The feature 'Amount' is the transaction Amount. This feature can be used for example-dependent cost-sensitive learning. Feature 'Class' is the response variable, and it takes value 1 in case of fraud and 0 otherwise.

Given the class imbalance ratio, it is recommended to measure the model accuracy using the Area Under the Precision-Recall Curve (AUPRC). Confusion matrix accuracy is not meaningful for unbalanced classification.

Exploratory Data Analysis and other findings

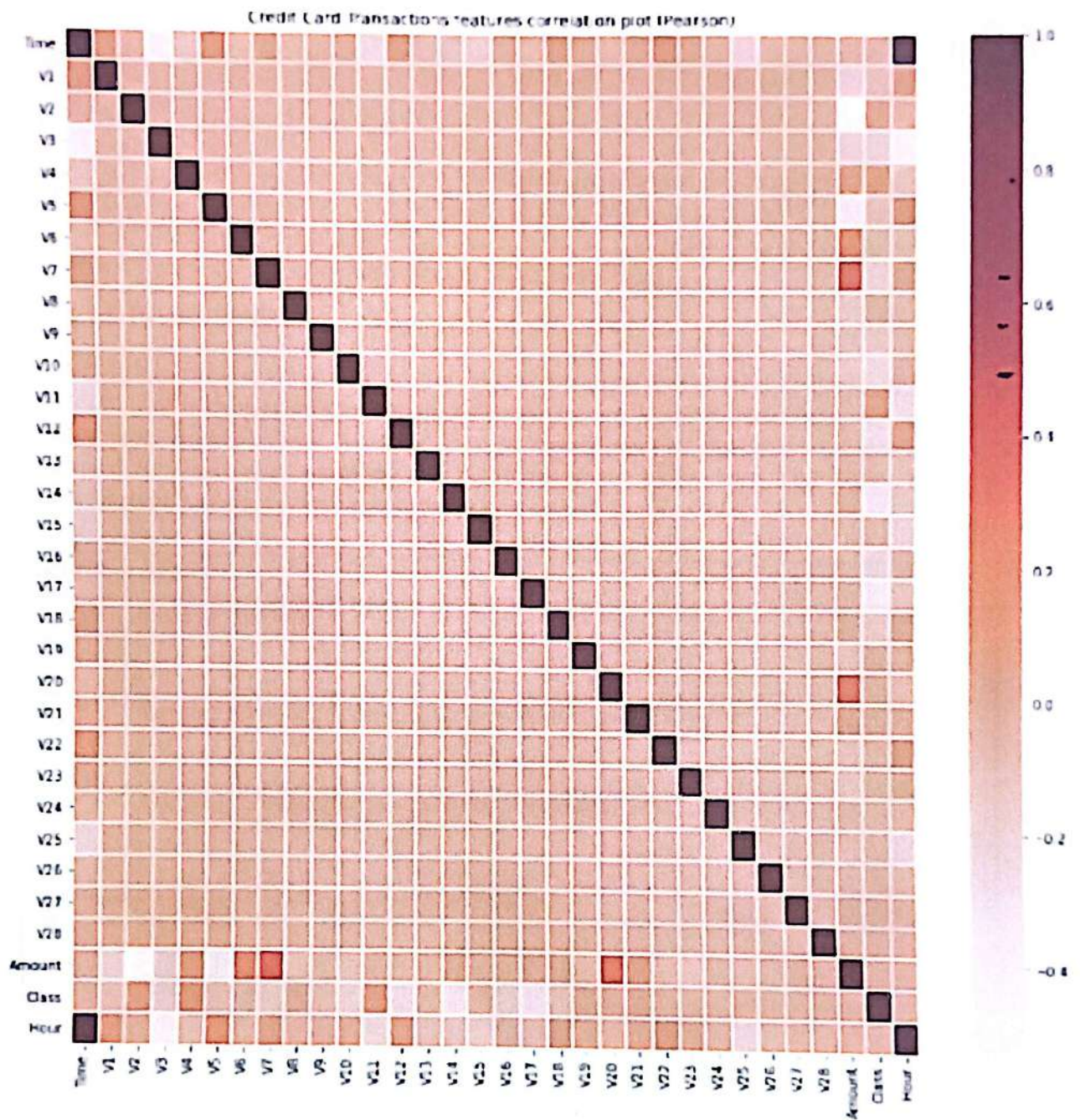
As noted earlier, the dataset is highly skewed, which can be seen from the below bar plot. Only 492 (or 0.172%) of the transactions are fraudulent. That means the data is highly unbalanced with respect to the target variable Class.

Credit Card Fraud Class - data unbalance (Not fraud = 0, Fraud = 1)



Correlation among independent variables

We can see that there is no correlation between the independent variables, and that is expected as the data providers have done PCA on the input variables except Time and Amount (PCA -> Principal component analysis). There are some small correlations between some of the independent variables and Time (inverse correlation with V3) and Amount (good correlation with V7 and V20, inverse correlation with V1 and V5). Multicollinearity occurs when independent variables in a regression model are correlated. This correlation is a problem because independent variables should be independent. If the degree of correlation between variables is high enough, it can cause problems when you fit the model and interpret the results. The degree of association is measured by a correlation coefficient, denoted by r . It is sometimes called Pearson's correlation coefficient after its originator and is a measure of linear association. If a curved line is needed to express the relationship, other and more complicated measures of the correlation must be used.



1.2 Problem Formulation

The problem formulation consists of just one sentence and should make it clear to everyone what research problem, you aim to address and to whom and where it is relevant.

Problem Formulation for our project:

Are the existing techniques used for detecting credit card frauds correctly and accurately providing efficient results or can it be improved using Machine Learning? Thus, our project tries to predict credit-card frauds using Machine Learning as it is believed to provide better results as compared to the existing techniques used to detect these frauds.

1.3 Proposed System

These are the proposed techniques used in this paper for detecting the frauds in credit card system. The comparison are made for different machine learning algorithms such as Logistic Regression, Decision Trees, Random Forest, to determine which algorithm suits best and can be adapted by credit card merchants for identifying fraud transactions. The Figure shows the architectural diagram for representing the overall system framework.[5]

Algorithm steps:	
Step 1:	Read the Dataset.
Step 2:	Random Sampling is done on the data set to make it balanced.
Step 3:	Divide the dataset into two parts i.e., Train dataset and Test dataset.
Step 4:	Accuracy and performance metrics has been calculated to know the efficiency for different algorithms.
Step 5:	Then retrieve the best algorithm based on efficiency for the given dataset.

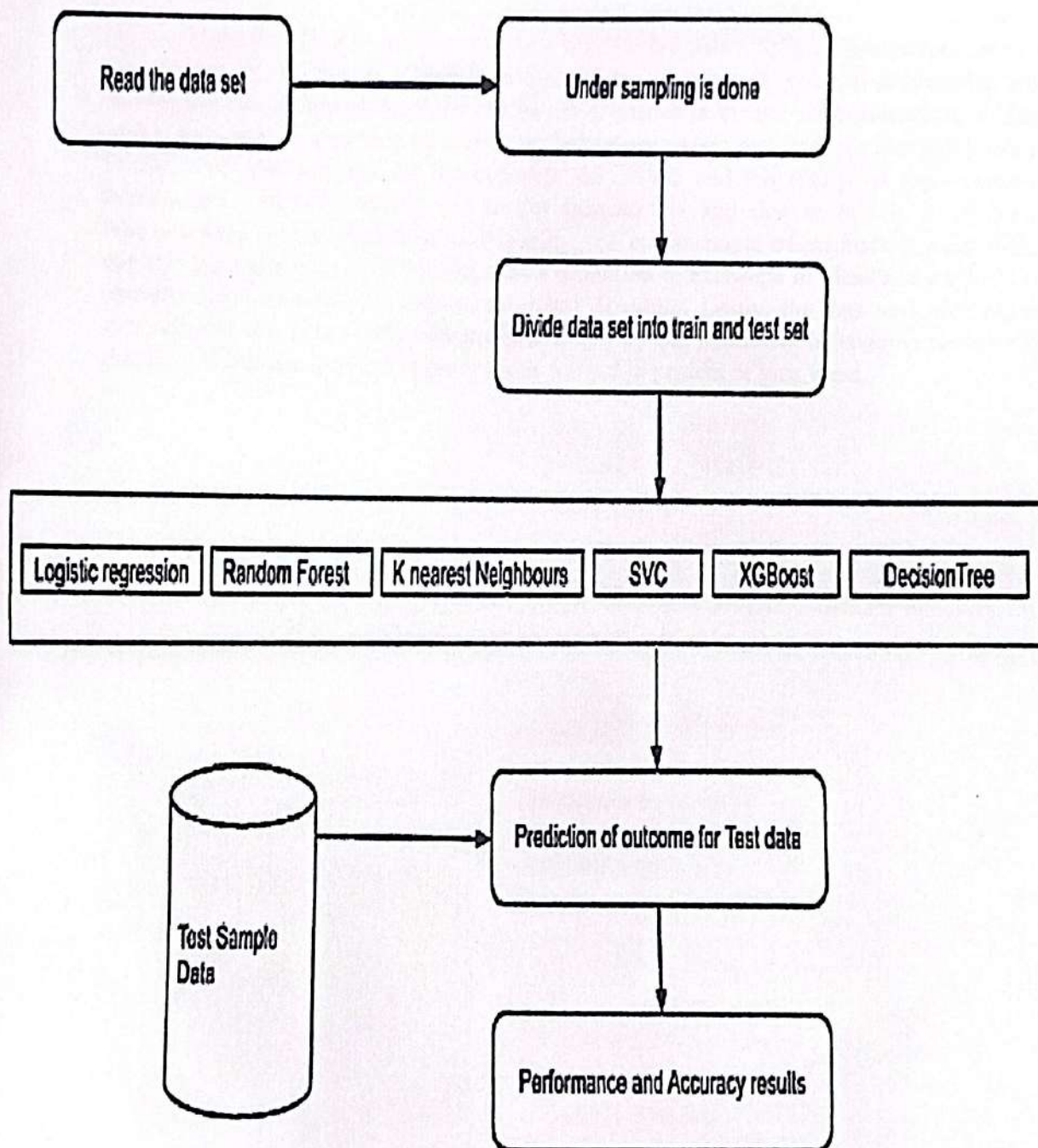


Figure 1: System Architecture

1.4 Scope of the Project

Credit card fraud detection is a very popular but also a difficult problem to solve. Firstly, due to issue of having only a limited amount of data, credit card makes it challenging to match a pattern for dataset. Secondly, there can be many entries in dataset with truncations of fraudsters which also will fit a pattern of legitimate behaviour. Also, the problem has many constraints. Firstly, data sets are not easily accessible for public and the results of researches are often hidden and censored, making the results inaccessible and due to this it is challenging to benchmarking for the models built. Secondly, the improvement of methods is more difficult by the fact that the security concern imposes a limitation to exchange of ideas and methods in fraud detection, and especially in credit card fraud detection. Lastly, the data sets are continuously evolving and changing making the profiles of normal and fraudulent behaviours always different, that is, a legit transaction in the past may be a fraud in present or vice versa.

CHAPTER 2

LITERATURE SERVEY

2. Literature Survey

Credit card fraud detection has drawn a lot of research interest and a number of techniques, with special emphasis on neural networks, data mining and distributed data mining have been suggested.

There have been various studies on credit card fraud detection. Machine learning and related methods are most commonly used, which include artificial neural networks, rule-induction techniques, decision trees, logistic regression, and support vector machines[1]

Behrouz, Emad and Sahil [2] have proposed credit card fraud detection using supervised machine learning algorithms. They have employed these algorithms to implement a super classifier using ensemble learning methods. Also, they compared and discussed the performance of various supervised machine learning algorithms that exist in literature against the super classifier that they implemented.

Navanshu Khare and Saad Yunus Sait[3] have explained the mathematics and working of every algorithm we have used in this paper.

Siddhartha Bhattacharyya and 4 others [6] in their paper in 2011 did a detailed comparative study of Support vector machine and random forest along with logistic regression. They concluded through experiments that Random Forest technique shows most accuracy followed by Logistic Regression and Support Vector Machine.

Y. Sahin and E. Duman [7] proposed fraud detection in credit card using a combination of Support Vector Machines and Decision Trees. Decision Trees outperformed SVMs when the size of data set was small but with increase in size of dataset SVM reached accuracy over decision trees.

CHAPTER 3

SYSTEM DESIGN

3. System Design

3.1 Functional Requirements

- The model should be able to give accurate and trustworthy predictions.
- The application must show graphical visualization of the predicted results and data in general.
- The user should be able to enter the input values for prediction.

3.2 Non Functional Requirements

Non Functional Requirements will describe how a system should behave and what limits are constrained on its functionality. It generally specifies the system's quality attributes or characteristics.

- **Availability:** The system should be available to any transaction verification system.
- **Correctness:** The accuracy of the system should be as maximum as possible for better prediction.
- **Maintainability:** The system should maintain correct history of records.
- **Usability:** The system should satisfy a maximum number of banking system needs.

3.3 Specific Requirements

Software requirements

- | | |
|----------------------|-------------------------------|
| 1. Languages | : Python-3 |
| 2. Operating Systems | : Windows, Linux, etc. |
| 3. Back End Software | : Anaconda, Jupyter notebook. |

Hardware Requirements:

- | | |
|--------------------|---------------------------|
| 1. CPU | : Intel Pentium IV 600MHz |
| 2. Hard disk space | : 20 GB or more |
| 3. Memory | : 4 GB RAM |

3.4 Block Diagram, Use Case, class diagram, etc

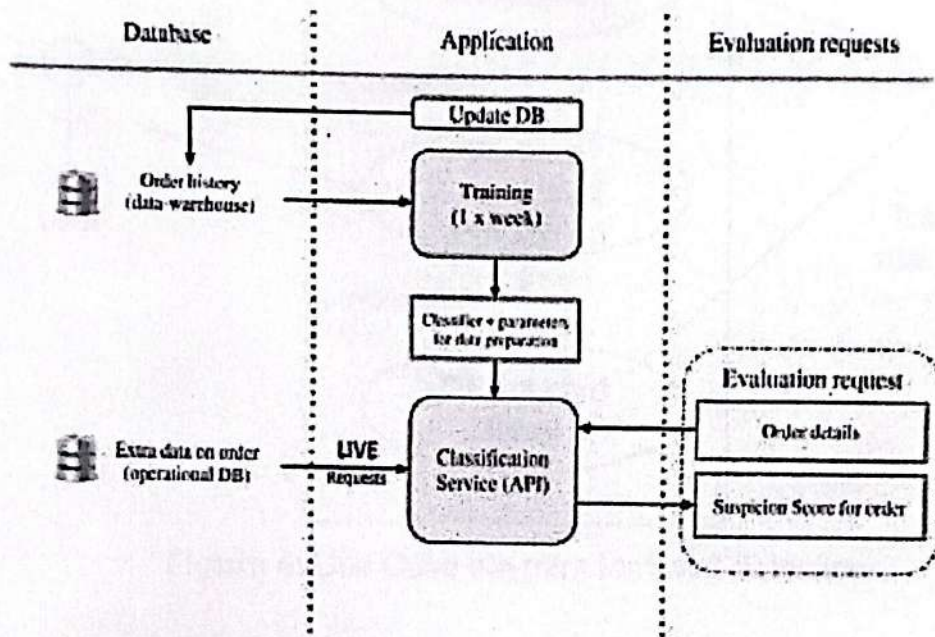


Figure 2: Activity diagram

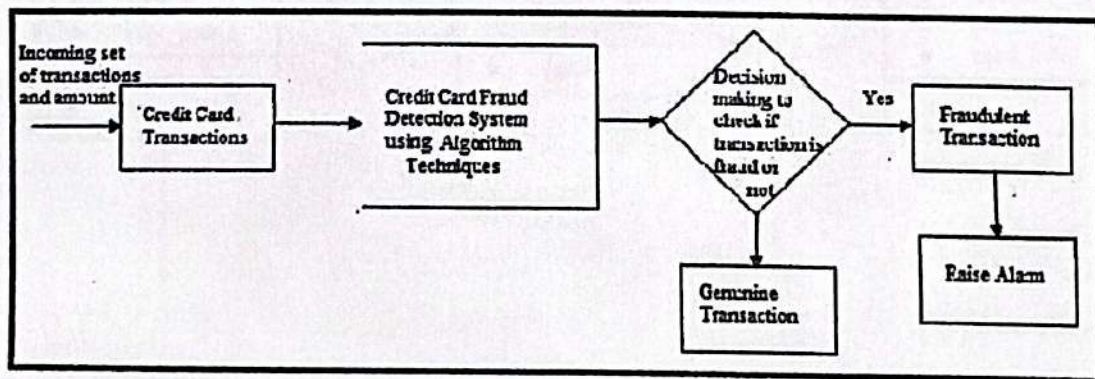


Figure 3: Flow Diagram

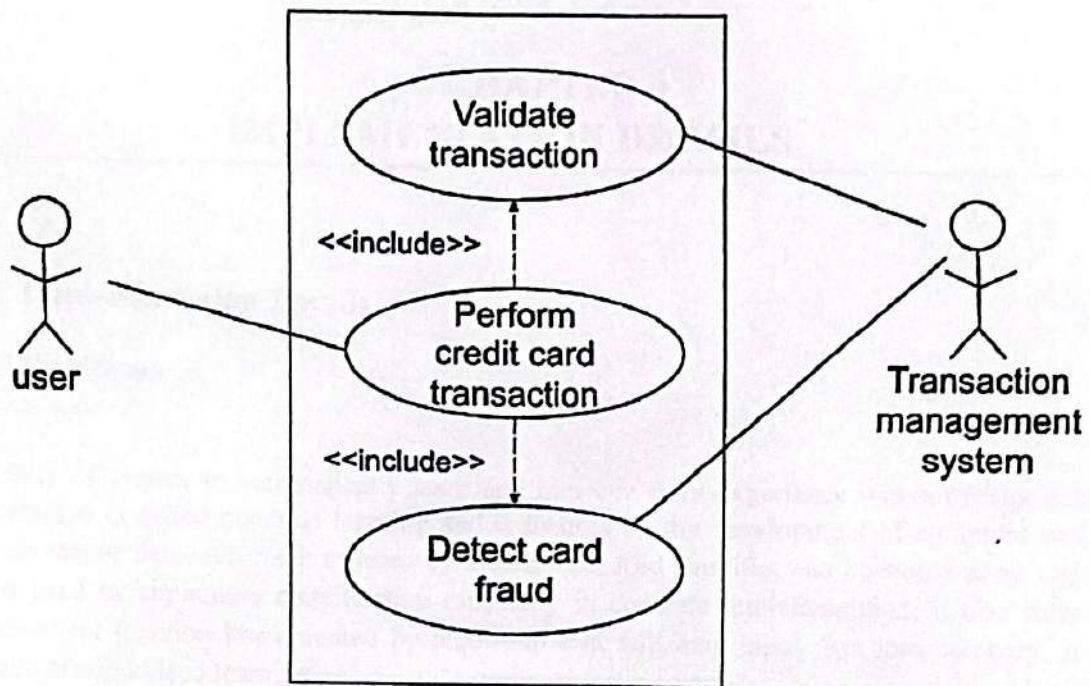


Figure 4: Use Case diagram for fraud detection

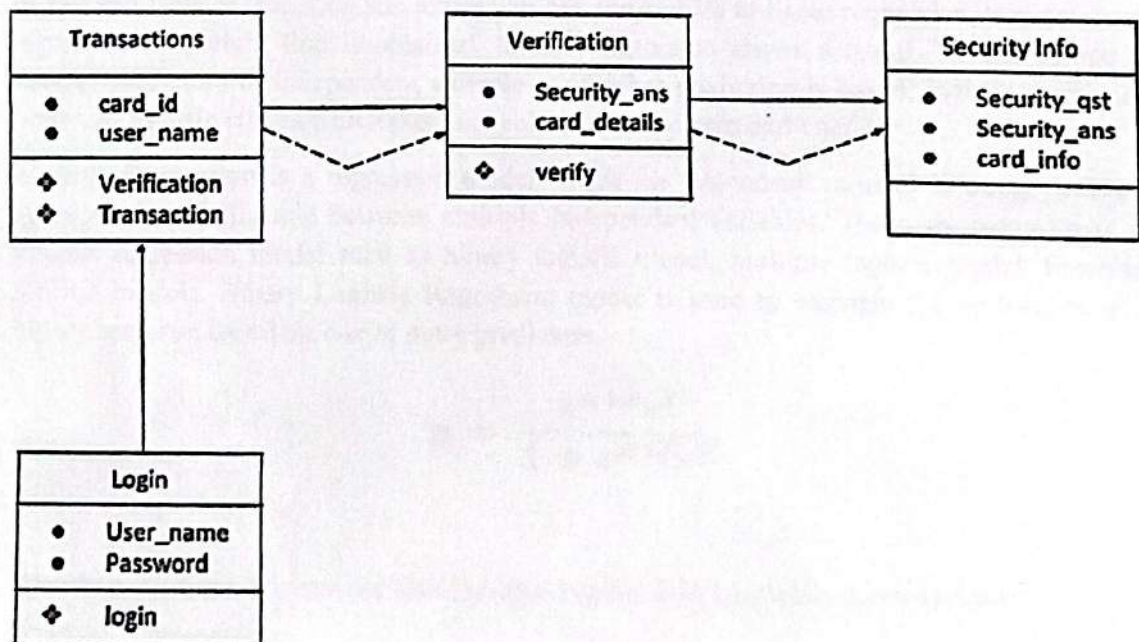


Figure 5: Class Diagram

CHAPTER 4

IMPLEMENTATION DETAILS

4. Implementation Details

4.1 Algorithms

Ability of system to automatically learn and improve from experience without being explicitly programmed is called machine learning and it focuses on the development of computer programs that can access data and use it to learn by themselves. And classifier can be stated as an algorithm that is used to implement classification especially in concrete implementation, it also refers to a mathematical function implemented by algorithm that will map input data into category. It is an instance of supervised learning

i.e. where training set of correctly identified observations is available.

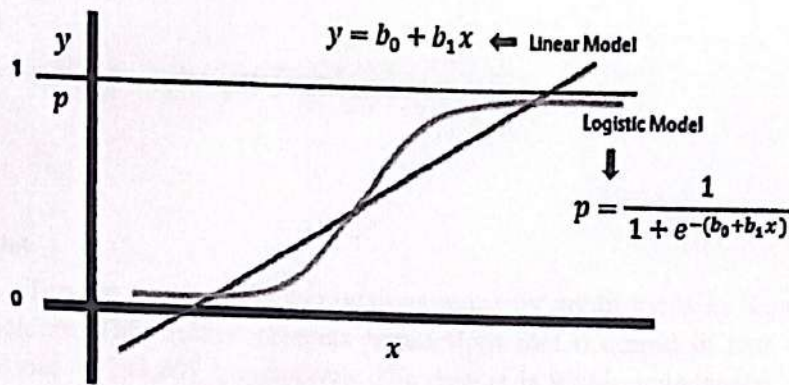
A. Logistic Regression

Logistic Regression is a supervised classification method that returns the probability of binary dependent variable that is predicted from the independent variable of dataset i.e. logistic regression predicts the probability of an outcome which has two values, either zero or one, no or yes and false or true. Logistic regression has similarities to linear regression, but, in linear regression a straight line is obtained, logistic regression shows a curve. The use of one or several predictors or independent variable is on what prediction is based, logistic regression produces logistic curves which plots the values between zero and one.

Logistic Regression is a regression model where the dependent variable is categorical and analyzes the relationship between multiple independent variables. There are many types of logistic regression model such as binary logistic model, multiple logistic model, binomial logistic models. Binary Logistic Regression model is used to estimate the probability of a binary response based on one or more predictors.

$$p = \frac{e^{\alpha + \beta_n X}}{1 + e^{\alpha + \beta_n X}}$$

Above equation represents the logistic regression in mathematical form.



B. Random Forest

Random Forest is an algorithm for classification and regression. Summarily, it is a collection of decision tree classifiers. Random forest has advantage over decision tree as it corrects the habit of overfitting to their training set. A subset of the training set is sampled randomly so that to train each individual tree and then a decision tree is built, each node then splits on a feature selected from a random subset of the full feature set. Even for large data sets with many features and data instances training is extremely fast in random forest and because each tree is trained independently of the others. The Random Forest algorithm has been found to provides a good estimate of the generalization error and to be resistant to overfitting. Random forest ranks the importance of variables in a regression or classification problem in a natural way can be done by Random Forest. Random forest has nearly the same hyperparameters as a decision tree or a bagging classifier. Fortunately, there's no need to combine a decision tree with a bagging classifier because you can easily use the classifier-class of random forest. With random forest, you can also deal with regression tasks by using the algorithm's regressor Random forest adds additional randomness to the model, while growing the trees. Instead of searching for the most important feature while splitting a node, it searches for the a random subset of features. This results in a wide diversity that generally results in a better model.

4.2 Working of the project

Dataset

The datasets contain transactions made by credit cards in September 2013 by European cardholders. This dataset presents transactions that occurred in two days, where we have 492 frauds out of 284,807 transactions. The dataset is highly unbalanced, the positive class (frauds) account for 0.172% of all transactions.

It contains only numeric input variables which are the result of a PCA transformation. Unfortunately, due to confidentiality issues, we cannot provide the original features and more background information about the data. Features V1, V2, ... V28 are the principal components obtained with PCA, the only features which have not been transformed with PCA are 'Time' and 'Amount'. Feature 'Time' contains the seconds elapsed between each transaction and the first transaction in the dataset. The feature 'Amount' is the transaction Amount, this feature can be used for example-dependant cost-sensitive learning. Feature 'Class' is the response variable and it takes value 1 in case of fraud and 0 otherwise. There are no "Null" values, so we don't have to work on ways to replace values.

Credit card dataset is highly imbalanced dataset because it carries more legitimate transactions as compared to the fraudulent one. That means prediction will get very high accuracy score without detecting a fraud transaction. When we apply any classification algorithm directly on the dataset we get the following results. Credit card dataset is highly imbalanced dataset because it carries more legitimate transactions as compared to the fraudulent one. That means prediction will get very high accuracy score without detecting a fraud transaction. When we apply any classification algorithm directly on the dataset we get the following results

first transaction in the dataset. The feature 'Amount' is the transaction Amount, this feature can be used for example-dependant cost-sensitive learning. Feature 'Class' is the response variable and it takes value 1 in case of fraud and 0 otherwise. There are no "Null" values, so we don't have to work on ways to replace values.

first transaction in the dataset. The feature 'Amount' is the transaction Amount, this feature can be used for example-dependant cost-sensitive learning. Feature 'Class' is the response variable and it takes value 1 in case of fraud and 0 otherwise. There are no "Null" values, so we don't have to work on ways to replace values.

The feature 'Amount' is the transaction Amount, this feature can be used for example-dependant cost-sensitive learning. Feature 'Class' is the response variable and it takes value 1 in case of fraud and 0 otherwise. There are no "Null" values, so we don't have to work on ways to replace values.

```
....: dataset.describe()
Out[1]:
```

	Time	V1	...	Amount	Class
count	284807.000000	2.848070e+05	...	284807.000000	284807.000000
mean	94813.859575	3.919560e-15	...	88.349619	0.001727
std	47488.145955	1.958696e+00	...	250.120109	0.041527
min	0.000000	-5.640751e+01	...	0.000000	0.000000
25%	54201.500000	-9.203734e-01	...	5.600000	0.000000
50%	84692.000000	1.810880e-02	...	22.000000	0.000000
75%	139320.500000	1.315642e+00	...	77.165000	0.000000
max	172792.000000	2.454930e+00	...	25691.160000	1.000000

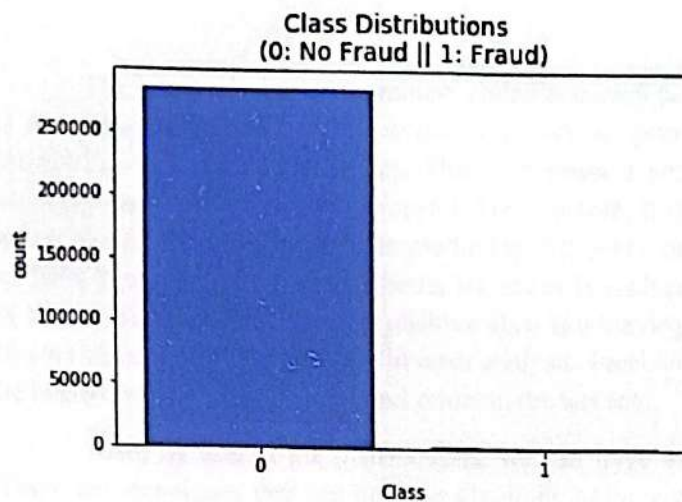
```
In [2]: dataset.head()
```

```
Out[2]:
```

	Time	V1	V2	V3	...	V27	V28	Amount	Class
0	0.0	-1.359807	-0.072781	2.536347	...	0.133558	-0.021053	149.62	0
1	0.0	1.191857	0.266151	0.166480	...	-0.008983	0.014724	2.69	0
2	1.0	-1.358354	-1.340163	1.773209	...	-0.055353	-0.059752	378.66	0
3	1.0	-0.966272	-0.185226	1.792993	...	0.062723	0.061458	123.50	0
4	2.0	-1.158233	0.877737	1.548718	...	0.219422	0.215153	69.99	0

Given the class imbalance ratio, we recommend measuring the accuracy using the Area Under the Precision-Recall Curve (AUPRC). Confusion matrix accuracy is not meaningful for unbalanced classification.

Why directly running prediction on the dataset is not a good idea?



No frauds 99.83% of dataset Frauds 0.17% of dataset

Notice how imbalanced is our original dataset is! Most of the transactions are non-fraud. If we use this data frame as the base for our predictive models and analysis, we might get a lot of errors and our algorithms will probably overfit since it will "assume" that most transactions are not fraud. But we don't want our model to assume, we want our model to detect patterns that give signs of fraud!

Credit card dataset is highly imbalanced dataset because it carries more legitimate transactions as compared to the fraudulent one. That means prediction will get very high accuracy score without detecting a fraud transaction. When we apply any classification algorithm directly on the dataset we get the following results.

```
KNearest
[[56864  0]
 [  93  5]]
accuracy_score: 0.9983673326077034
miscalssification: 0.0016326673922966162
Sensitivity : 1.0
Specificity : 0.05102040816326531
precision_score: 1.0
f1_score: 0.09708737864077671
```

```
LogisticRegression
[[56853  11]
 [  46  52]]
accuracy_score: 0.9989993328885924
miscalssification: 0.0010006671114075605
Sensitivity : 0.9998065559932471
Specificity : 0.5306122448979592
precision_score: 0.8253968253968254
f1_score: 0.6459627329192548
```

```
XGBClassifier
[[56858  6]
 [  18  80]]
accuracy_score: 0.9995786664794073
miscalssification: 0.0004213335205927038
Sensitivity : 0.9998944850872257
Specificity : 0.8163265306122449
precision_score: 0.9302325581395349
f1_score: 0.8695652173913043
```

```
RandomForestClassifier
[[56862  2]
 [  21  77]]
accuracy_score: 0.9995962220427653
miscalssification: 0.00040377795723467447
Sensitivity : 0.9999648283624085
Specificity : 0.7857142857142857
precision_score: 0.9746835443037974
f1_score: 0.8700564971751412
```

The results deal with a problem called accuracy paradox. The ACCURACY PARADOX is the paradoxical finding that accuracy is not a good metric for predictive models when classifying in predictive analytics. This is because a simple model may have a high level of accuracy but be too crude to be useful. For example, if the incidence of category A is dominant, being found in 99% of cases, then predicting that every case is category A will have an accuracy of 99%. Precision and recall are better measures in such cases. The underlying issue is that there is a class imbalance between the positive class and the negative class. Prior probabilities for these classes need to be accounted for in error analysis. Precision and recall help, but precision too can be biased by very unbalanced class priors in the test sets.

Also, by seeing the distributions, we can have an idea how skewed are these features. There are techniques that can help the distributions be less skewed which will be implemented in this report in the future.

Credit card dataset is highly imbalanced dataset because it carries more legitimate transactions as compared to the fraudulent one. That means prediction will get very high accuracy score without detecting a fraud transaction. When we apply any classification algorithm directly on the dataset we get the following results.

Credit card dataset is highly imbalanced dataset because it carries more legitimate transactions as compared to the fraudulent one. That means prediction will get very high accuracy score without detecting a fraud transaction. When we apply any classification algorithm directly on the dataset we get the following results. Credit card dataset is highly imbalanced dataset because it carries more legitimate transactions as compared to the fraudulent one. That means prediction will get very high accuracy score without detecting a fraud transaction. When we apply any classification algorithm directly on the dataset we get the following results

first transaction in the dataset. The feature 'Amount' is the transaction Amount, this feature can be used for example-dependant cost-sensitive learning. Feature 'Class' is the response variable and it takes value 1 in case of fraud and 0 otherwise. There are no "Null" values, so we don't have to work on ways to replace values.

Notice how imbalanced is our original dataset is! Most of the transactions are non-fraud. If we use this data frame as the base for our predictive models and analysis, we might get a lot of errors and our algorithms will probably overfit since it will "assume" that most transactions are not fraud. But we don't want our model to assume, we want our model to detect patterns that give signs of fraud!

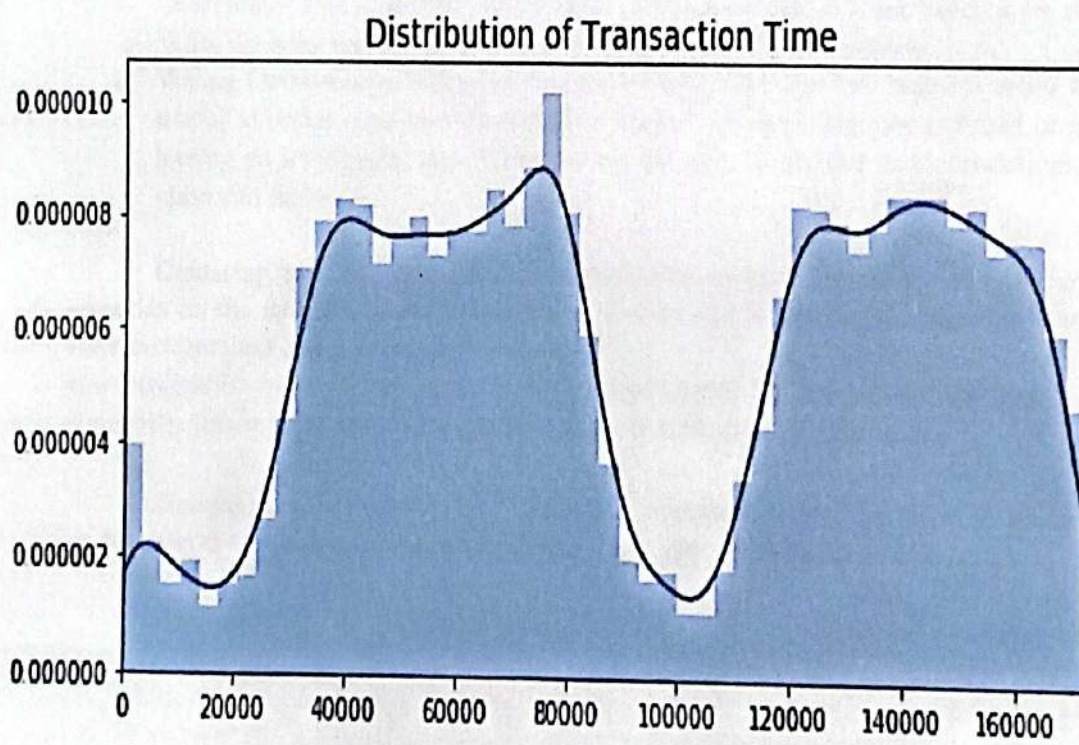
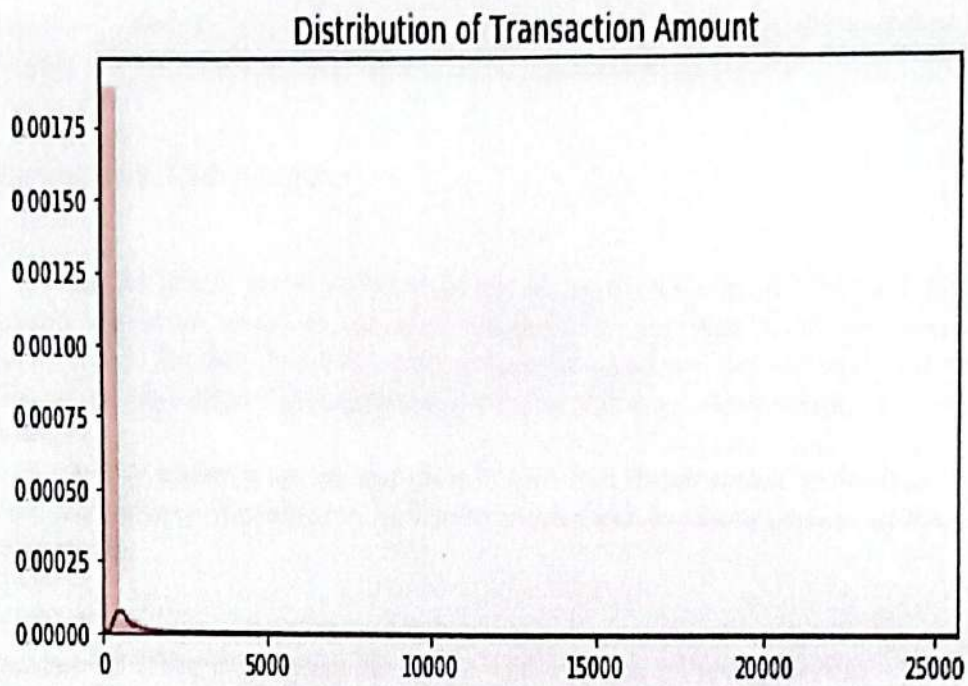


Figure 9: Distribution of Transaction Amount and Time

To handle this kind of problem one better way is to class distribution, i.e., sampling minority classes. In sampling minority, class training example can be increased in proportion to the majority class to raise the chance of correct prediction by the algorithm.

Scaling and Distributing:

In this phase, we will first scale the columns comprise of Time and Amount. Time and amount should be scaled as the other columns. On the other hand, we need to also create a subsample of the data frame in order to have an equal amount of Fraud and Non-Fraud cases, helping our algorithms better understand patterns that determines whether a transaction is a fraud or not.

In this scenario, our subsample will be a data frame with a 50/50 ratio of fraud and non-fraud transactions. Meaning our sub-sample will have the same amount of fraud and non-fraud transactions.

We need to create a sub sample because we saw that the original data frame was heavily imbalanced! Using the original data frame will cause the following issues:

1. **Overfitting:** Our classification models will assume that in most cases there are no frauds! What we want for our model is to be certain when a fraud occurs.
2. **Wrong Correlations:** Although we don't know what the "V" features stand for, it will be useful to understand how each of this feature influence the result (Fraud or No Fraud) by having an imbalance data frame we are not able to see the true correlations between the class and features.

Cantering and scaling happen independently on each feature by computing the relevant statistics on the samples in the training set. Median and interquartile range are then stored to be used on later data using the transform method.

Standardization of a dataset is a common requirement for many machine learning estimators. Typically, this is done by removing the mean and scaling to unit variance.

However, outliers can often influence the sample mean / variance in a negative way. In such cases, the median and the interquartile range often give better results.

As RobustScaler, QuantileTransformer is robust to outliers in the sense that adding or removing outliers in the training set will yield approximately the same transformation on held out data. But contrary to RobustScaler, QuantileTransformer will also automatically collapse any outlier by setting them to the a priori defined range boundaries (0 and 1).

Splitting the Data (Original DataFrame)

Summary:

- There are 492 cases of fraud in our dataset so we can randomly get 492 cases of non-fraud to create our new sub data frame.
- We concatenate the 492 cases of fraud and non-fraud, creating a new sub-sample.

Scaled amount and scaled time are the columns with scaled values:

```
from sklearn.preprocessing import StandardScaler, RobustScaler
rob_scaler = RobustScaler()
dataset['scaled_amount'] = rob_scaler.fit_transform(dataset['Amount'].values.reshape(-1,1))
dataset['scaled_time'] = rob_scaler.fit_transform(dataset['Time'].values.reshape(-1,1))
dataset.drop(['Time', 'Amount'], axis=1, inplace=True)
scaled_amount = dataset['scaled_amount']
scaled_time = dataset['scaled_time']
dataset.drop(['scaled_amount', 'scaled_time'], axis=1, inplace=True)
dataset.insert(0, 'scaled_amount', scaled_amount)
dataset.insert(1, 'scaled_time', scaled_time)
# Amount and Time are Scaled!
```

In [6]: dataset.head()

Out[6]:

	scaled_amount	scaled_time	V1	...	V27	V28	Class
42948	-0.167680	-0.509675	-0.256652	...	-0.547145	-0.380498	0
74625	-0.293579	-0.340923	-0.357039	...	0.243853	0.085171	0
236178	-0.296793	0.751959	-0.443426	...	-0.665420	-0.268262	0
192669	0.041920	0.529940	2.334822	...	0.006936	-0.071813	0
15736	1.089779	-0.675866	-23.914101	...	1.458076	0.430315	1

Before proceeding with the Random Under Sampling technique we have to separate the original data frame for testing purposes, although we are splitting the data when implementing Random Under Sampling or Oversampling techniques, we want to test our models on the original testing set, not on the testing set created by either of these techniques. The main goal is to fit the model either with the data frames that were under sample and oversample (in order for our models to detect the patterns), and test it on the original testing set.

One of the most common and simplest strategies to handle imbalanced data is to under sample the majority class.

Oversampling the minority class can result in overfitting problems if we oversample before cross-validating.

```
In [8]: X = dataset.drop('Class', axis=1)
...: y = dataset[['Class']]
...: from sklearn.model_selection import StratifiedShuffleSplit
...: sss = StratifiedShuffleSplit(n_splits=5, test_size=0.2, random_state=42)
...: for train_index, test_index in sss.split(X, y):
...:     print("Train:", train_index, "Test:", test_index)
...:     original_Xtrain, original_Xtest = X.iloc[train_index], X.iloc[test_index]
...:     original_ytrain, original_ytest = y.iloc[train_index], y.iloc[test_index]
...:
...:     original_Xtrain = original_Xtrain.values
...:     original_Xtest = original_Xtest.values
...:     original_ytrain = original_ytrain.values
...:     original_ytest = original_ytest.values
...:     train_unique_label, train_counts_label = np.unique(original_ytrain,
return_counts=True)
...:     test_unique_label, test_counts_label = np.unique(original_ytest,
return_counts=True)
...:     print('-' * 100)
...:     print('Label Distributions: \n')
...:     print(train_counts_label/ len(original_ytrain))
...:     print(test_counts_label/ len(original_ytest))
```

```

Train: [265518 180305 42664 ... 29062 13766 17677] Test: [263020 11378 147283 ... 274532 269819 64170]
Train: [ 72227 114282 16818 ... 264471 191914 284017] Test: [202638 32978 128121 ... 244024 127667 48318]
Train: [ 20895 114622 167683 ... 244502 178972 218506] Test: [284352 82483 90981 ... 171224 168807 271602]
Train: [122248 181660 194400 ... 104631 277586 29432] Test: [225673 63348 68025 ... 279451 77554 76043]
Train: [241684 223467 136928 ... 86495 160550 49633] Test: [157557 204860 83760 ... 251478 178967 216850]

```

Label Distributions:

```

[0.99827075 0.00172925]
[0.99827955 0.00172045]

```

Under Sampling :

In this phase of the project we will implement "*Under Sampling*" which basically consists of removing data in order to have a more balanced dataset and thus avoiding our models to overfitting.[4]

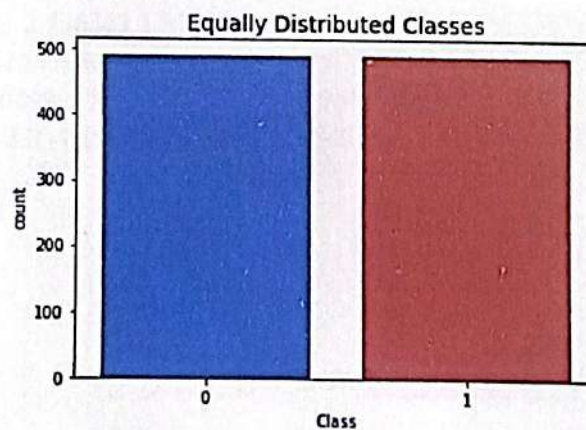
Due to recent advancements in communication technology and e-commerce systems, credit cards are currently the most popular form of payment for online as well as offline purchases. As a consequence, there is substantially more fraud involved in the transaction processes. Data mining has an issue for prediction, and data classification, therefore finding occurrences is essential. Unusual events are challenging to be identified due to their irregularity and casualness, but misclassifying unusual events can lead to large financial losses. In this study, an approach for detecting credit card fraud using machine learning methods, such as K-Nearest Neighbors, random forest, decision trees, logistic regression, and support vector machines, is proposed. The research attempts to look at the effectiveness of the classification models while applying both the Oversampling and Undersampling techniques to find instances of fraud in the dataset for fraudulent activities. The experimental study used two days of credit card transactions made by European cardholders in September 2013. To evaluate the models' performances, confusion matrix, precision, recall, f1_score, cross-validation score, and ROC_AUC score metrics were used. From different experiments of the tested model, it can be easily observed that the performance of all models was better compared with previous literature thus the KNN was the best in almost all metrics used.

- The first thing we have to do is determine how imbalanced is our class (use "value_counts()" on the class column to determine the amount for each label)
- Once we determine how many instances are considered fraud transactions (Fraud = "1") , we should bring the non-fraud transactions to the same amount as fraud transactions (assuming we want a 50/50 ratio), this will be equivalent to 492 cases of fraud and 492 cases of non-fraud transactions.
- After implementing this technique, we have a sub-sample of our data frame with a 50/50 ratio with regards to our classes. Then the next step we will implement isto shuffle the data to see if our models can maintain a certain accuracy every time, werun this script.

Note: The main issue with "Random Under-Sampling" is that we run the risk that our classification models will not perform as accurate as we would like to since there is a great deal of information loss (bringing 492 non-fraud transaction from 284,315 non-fraud transaction)

```
In [9]: dataset = dataset.sample(frac=1)
...: # amount of fraud classes 492 rows.
...: fraud_df = dataset.loc[dataset['Class'] == 1]
...: non_fraud_df = dataset.loc[dataset['Class'] == 0][:497]
...: normal_distributed_df = pd.concat([fraud_df, non_fraud_df])
...: # Shuffle dataframe rows
...: new_df = normal_distributed_df.sample(frac=1, random_state=42)
...: new_df.head()
...: new_df.describe()
...: print('Distribution of the Classes in the subsample dataset')
...: print(new_df['Class'].value_counts()/len(new_df))
Distribution of the Classes in the subsample dataset
0    0.502528
1    0.497472
Name: Class, dtype: float64
```

Now that we have our dataframe correctly balanced, we can go further with our **analysis** and data preprocessing.



Now that we have sampled our data set, we apply our supervised learning models.

Code

```
#Import modules
```

```
import numpy as np
import matplotlib.pyplot as plt import pandas as pd
import seaborn as sns
import warnings warnings.filterwarnings('ignore')

import seaborn as sns #..... from sklearn.preprocessing import
StandardScaler #..... from sklearn.model_selection import train_test_split,
cross_val_score #..... from sklearn.metrics import confusion_matrix, precision_score ,auc, roc_curve
#. import itertools #..... from sklearn import tree, linear_model,
neighbors, ensemble, discriminant_analys from sklearn.neighbors import KNeighborsClassifier
#.....
```

```
#Loading the dataset
```

```
df = pd.read_csv('creditcard.csv')
```

```
df.head()
```

```
Time V1 V2 V3 V4 V5 V6 V7
```

```
0 0.0 -1.359807 -0.072781 2.536347 1.378155 -0.338321 0.462388 0.239599 0.0 1 0.0 1.191857
0.266151 0.166480 0.448154 0.060018 -0.082361 -0.078803 0.0 2 1.0 -1.358354 -1.340163 1.773209
0.379780 -0.503198 1.800499 0.791461 0.2 3 1.0 -0.966272 -0.185226 1.792993 -0.863291 -0.010309
1.247203 0.237609 0.3 4 2.0 -1.158233 0.877737 1.548718 0.403034 -0.407193 0.095921 0.592941 -
0.2
```

```
5 rows × 31 columns
```

```
df.describe()
```

```
Time V1 V2 V3 V4
```

```
count 284807.000000
```

```
2.848070e+05 3.918649e-15 1.958696e+00 -5.640751e+01 -9.203734e-01 1.810880e-02
1.315642e+00 2.454930e+00
```

```
2.848070e+05 5.682686e-16 1.651309e+00 -7.271573e+01 -5.985499e-01 6.548556e-02 8.037239e-
01 2.205773e+01
```

```
2.848070e+05 -8.761736e-15 1.516255e+00
```

-4.832559e+01 -8.903648e-01 1.798463e-01 1.027196e+00 9.382558e+00

2.848070e+05 2.84 2.811118e-15 -1.5 1.415869e+00 1.38

-5.683171e+00 -1.13 -8.486401e-01 -6.9 -1.984653e-02 -5.4

7.433413e-01 6.1 1.687534e+01 3.48

mean std min 25% 50% 75% max

94813.859575 47488.145955 0.000000 54201.500000 84692.000000 139320.500000 172792.000000

8 rows × 31 columns

df.head()

Time V1 V2 V3 V4 V5 V6 V7

0 0.0 -1.359807 -0.072781 2.536347 1.378155 -0.338321 0.462388 0.239599 0.0 1 0.0 1.191857
0.266151 0.166480 0.448154 0.060018 -0.082361 -0.078803 0.0 2 1.0 -1.358354 -1.340163 1.773209
0.379780 -0.503198 1.800499 0.791461 0.2 3 1.0 -0.966272 -0.185226 1.792993 -0.863291 -0.010309
1.247203 0.237609 0.3 4 2.0 -1.158233 0.877737 1.548718 0.403034 -0.407193 0.095921 0.592941 -
0.2

5 rows × 31 columns

80
521
02
74
159
335
192
01
986
851
476
774
705

7 0

4 3

7 8 6

df.tail()

Time V1 V2 V3 V4 V5 V6

284802 172786.0 284803 172787.0 284804 172788.0 284805 172788.0 284806 172792.0

5 rows × 31 columns

-11.881118 10.071785 -9.834783 -2.066656 -5.364473 -2.606837 -4.9 -0.732789 -0.055080 2.035030
-0.738589 0.868229 1.058415 0.0 1.919565 -0.301254 -3.249640 -0.557828 2.630515 3.031260 -0.2 -
0.240440 0.530483 0.702510 0.689799 -0.377961 0.623708 -0.6 -0.533413 -0.189733 0.703337 -
0.506271 -0.012546 -0.649617 1.5

182

243

968

861

df.info()

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 284807 entries, 0 to 284806

Data columns (total 31 columns):

Column Non-Null Count Dtype

--- --
0 Time 284807 non-null float64

1V1

2V2

3V3

4V4

5V5

6V6

7V7

8V8

9V9

10 V10

11 V11

12 V12

13 V13

14 V14

15 V15

16 V16

17 V17

18 V18

19 V19

20 V20

21 V21

22 V22

```
23 V23
24 V24
25 V25
26 V26
27 V27
28 V28
29 Amount 284807 non-null float64 30 Class 284807 non-null int64
```

```
dtypes: float64(30), int64(1)
```

```
memory usage: 67.4 MB
```

```
#Exploratory Data Analysis
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
284807 non-null float64
```

```
sns.countplot(data=df,x="Class")
```

```
count = df["Class"].value_counts()
```

```
plt.yticks(count)
```

```
plt.show()
```

```
df_temp = df.drop(columns=['Time', 'Amount', 'Class'], axis=1)
```

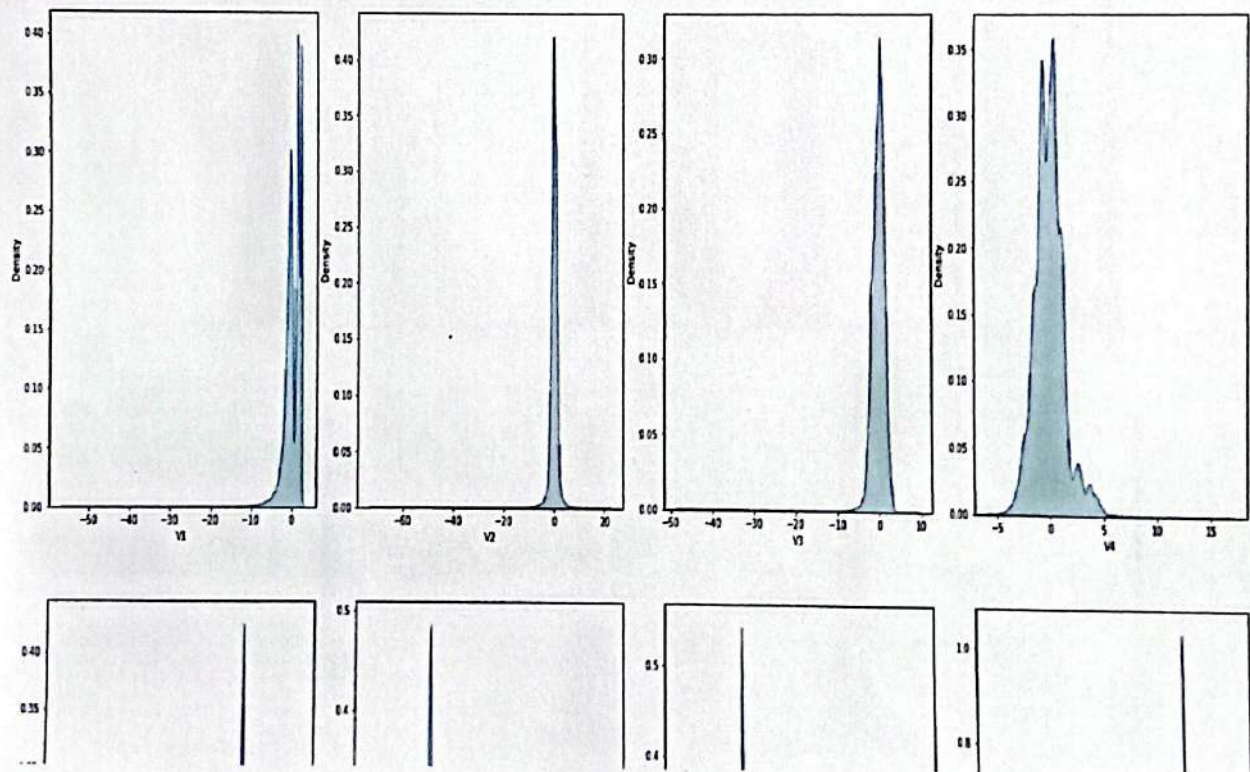
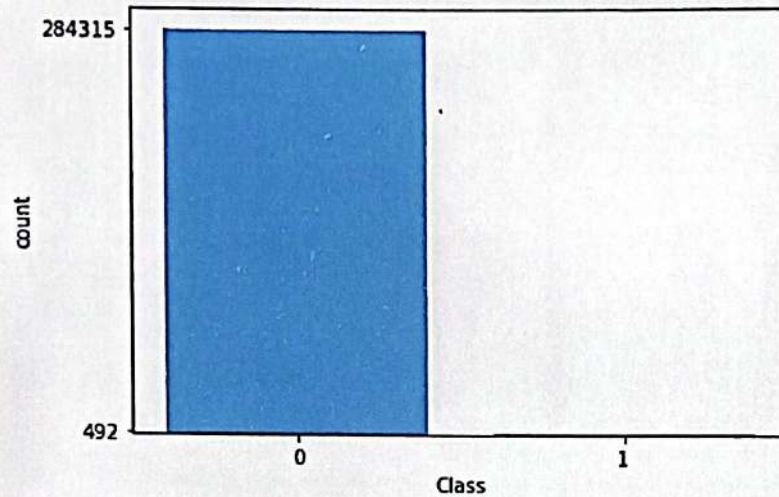
```
# create dist plots
```

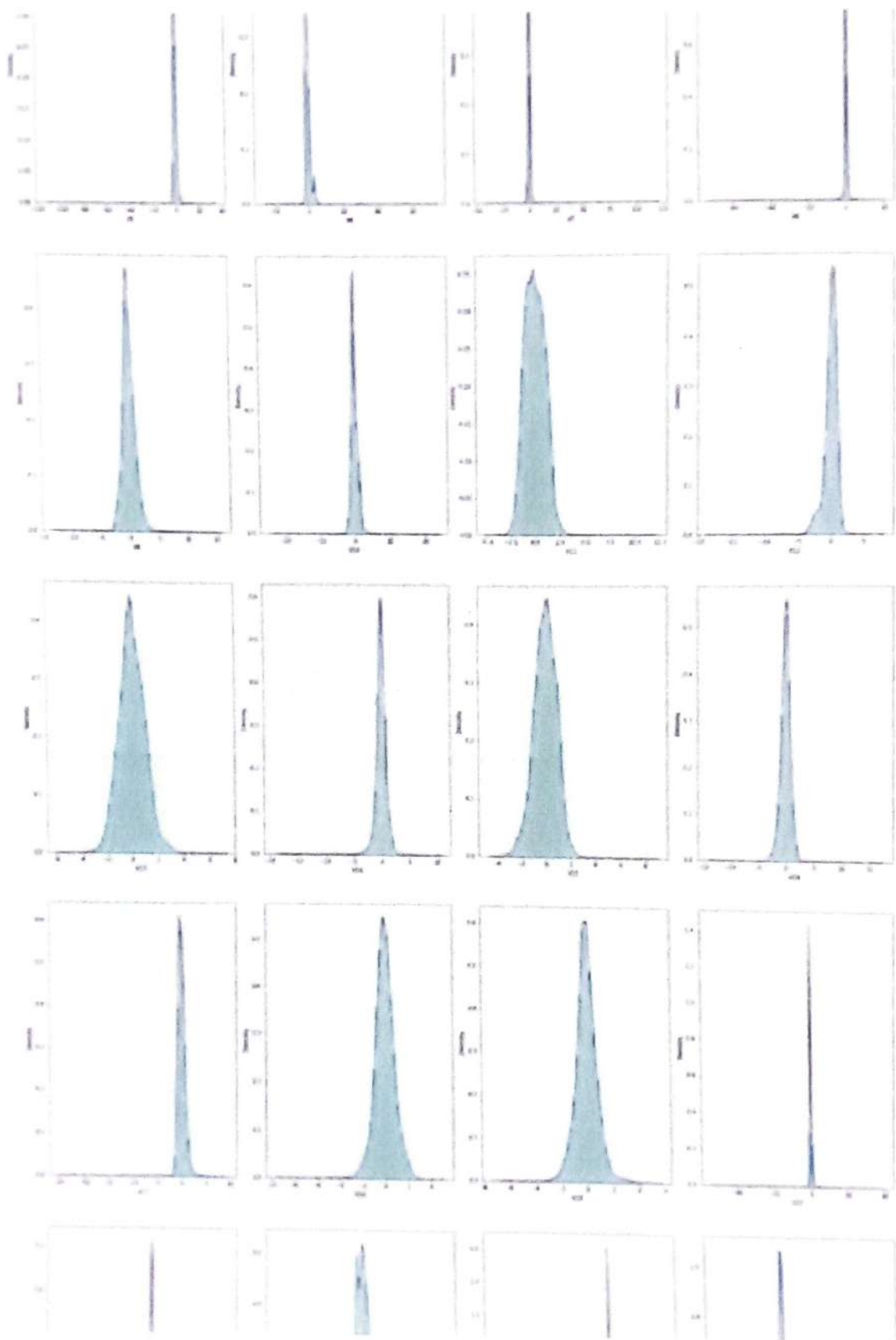
```
fig, ax = plt.subplots(ncols=4, nrows=7, figsize=(20, 50))
```

```

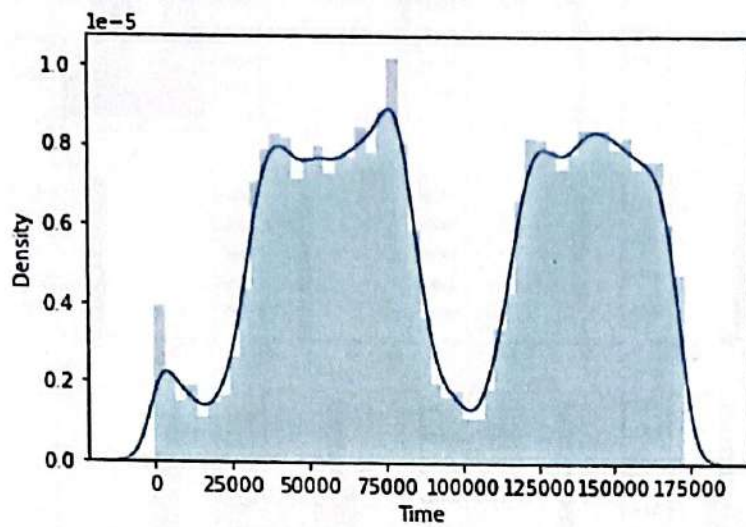
index = 0
ax = ax.flatten()
for col in df_temp.columns:
    sns.distplot(df_temp[col], ax=ax[index])
    index += 1
plt.tight_layout(pad=0.5, w_pad=0.5, h_pad=5)

```

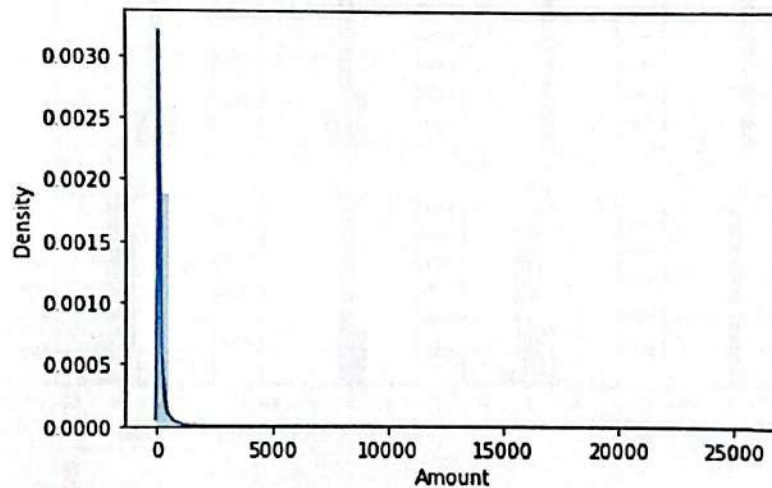




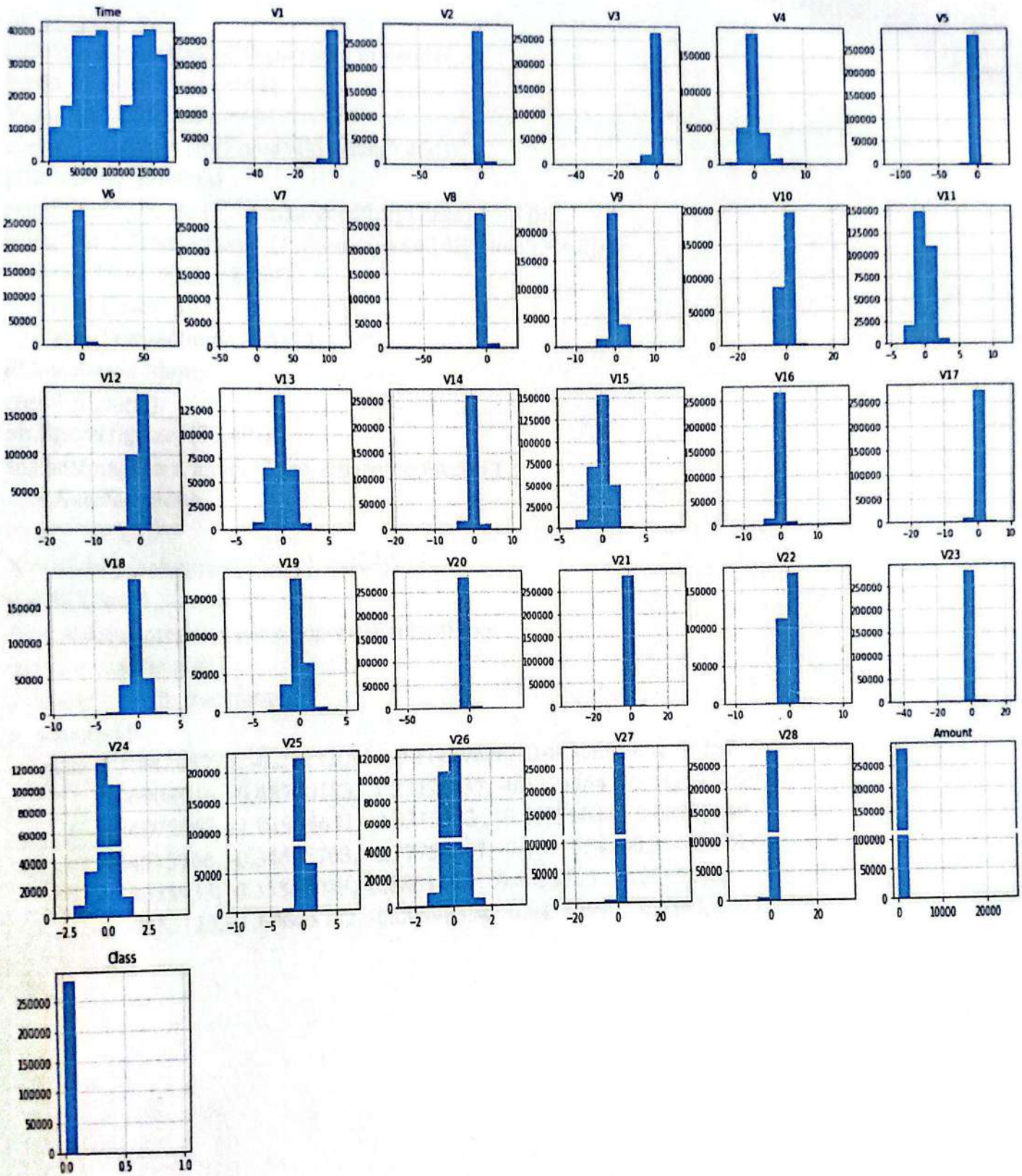
```
sns.distplot(df['Time'])
<AxesSubplot:xlabel='Time', ylabel='Density'>
```



```
sns.distplot(df['Amount'])
<AxesSubplot:xlabel='Amount', ylabel='Density'>
```



```
# Plot histograms of each parameter
df.hist(figsize = (20, 20))
plt.show()
```

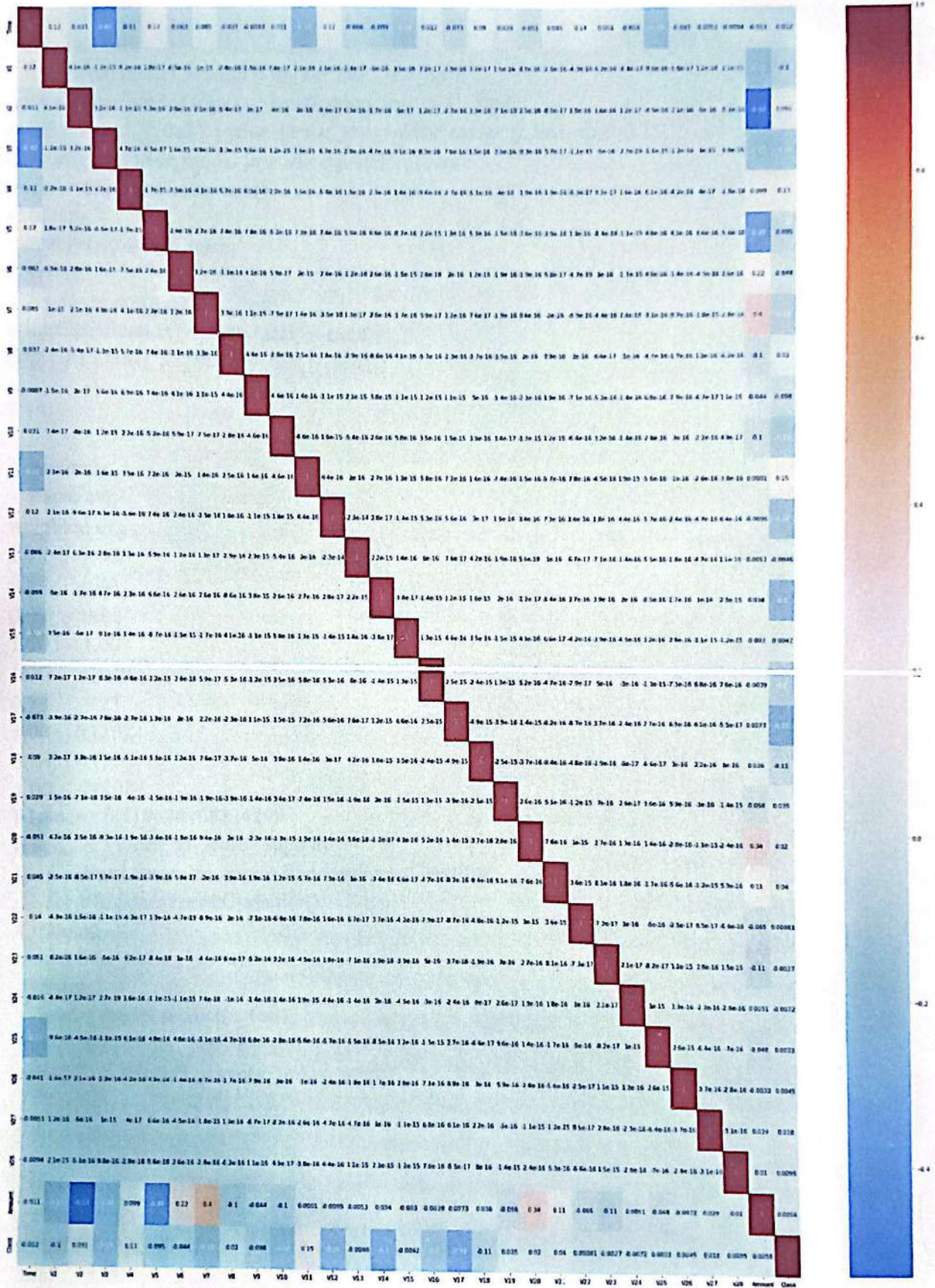


```

# Determine number of fraud cases in dataset
Fraud = df[df['Class'] == 1]
Valid = df[df['Class'] == 0]
outlier_fraction = len(Fraud)/float(len(Valid))
print(outlier_fraction)
print('Fraud Cases: {}'.format(len(df[df['Class'] == 1])))
print('Valid Transactions: {}'.format(len(df[df['Class'] == 0])))
0.0017304750013189597
Fraud Cases: 492
Valid Transactions: 284315
#Coorelation Matrix
corr = df.corr()
plt.figure(figsize=(30,40))
sns.heatmap(corr, annot=True, cmap='coolwarm')
<AxesSubplot:>

X = df.drop(columns=['Class'], axis=1)
y = df['Class']
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
x_scaler = sc.fit_transform(X)
x_scaler[-1]
array([ 1.64205773, -0.27233093, -0.11489898,  0.46386564, -0.35757 ,
        -0.00908946, -0.48760183,  1.27476937, -0.3471764 ,  0.44253246,
        -0.84072963, -1.01934641, -0.0315383 , -0.18898634, -0.08795849,
         0.04515766, -0.34535763, -0.77752147,  0.1997554 , -0.31462479,
         0.49673933,  0.35541083,  0.8861488 ,  0.6033653 ,  0.01452561,
        -0.90863123, -1.69685342, -0.00598394,  0.04134999,  0.51435531])

```



```

#Model Training
# train test split
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, f1_score
x_train, x_test, y_train, y_test = train_test_split(x_scaler, y, test_size=0.25,
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
# training
model.fit(x_train, y_train)
# testing
y_pred = model.predict(x_test)
print(classification_report(y_test, y_pred))
print("F1 Score:",f1_score(y_test, y_pred))

```

0 1

```

accuracy
macro avg
weighted avg
1.00 1.00
0.85 0.63
0.92 0.81
1.00 1.00
1.00 71079
0.72 123
1.00 71202
0.86 71202
1.00 71202

```

F1 Score: 0.719626168224299

```

precision recall f1-score support
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
# training
model.fit(x_train, y_train)
# testing
y_pred = model.predict(x_test)
print(classification_report(y_test, y_pred))
print("F1 Score:",f1_score(y_test, y_pred))

```

0 1

```

accuracy
macro avg
weighted avg
1.00 1.00
0.96 0.79

```

```

0.98  0.89
1.00  1.00
1.00  71079
0.87  123
1.00  71202
0.93  71202
1.00  71202

```

F1 Score: 0.8660714285714286

```
precision recall f1-score support
```

```
import matplotlib.pyplot as plt
```

```
import itertools
```

```
from sklearn import svm, datasets
```

```
from sklearn.metrics import confusion_matrix
```

```
def plot_confusion_matrix(cm, classes,
                           normalize=False,
                           title='Confusion matrix',
                           cmap=plt.cm.Blues):
```

```
    """
```

This function prints and plots the confusion matrix.

Normalization can be applied by setting `normalize=True`.

```
    """
```

```
    if normalize:
```

```
        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
```

```
        print("Normalized confusion matrix")
```

```
    else:
```

```
        print('Confusion matrix, without normalization')
```

```
    print(cm)
```

```
    plt.imshow(cm, interpolation='nearest', cmap=cmap)
```

```
    plt.title(title)
```

```
    plt.colorbar()
```

```
    tick_marks = np.arange(len(classes))
```

```
    plt.xticks(tick_marks, classes, rotation=45)
```

```
    plt.yticks(tick_marks, classes)
```

```
    fmt = '.2f' if normalize else 'd'
```

```
    thresh = cm.max() / 2.
```

```
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
```

```
        plt.text(j, i, format(cm[i, j], fmt),
```

```
                horizontalalignment="center",
```

```
                color="white" if cm[i, j] > thresh else "black")
```

```
    plt.ylabel('True label')
```

```
    plt.xlabel('Predicted label')
```

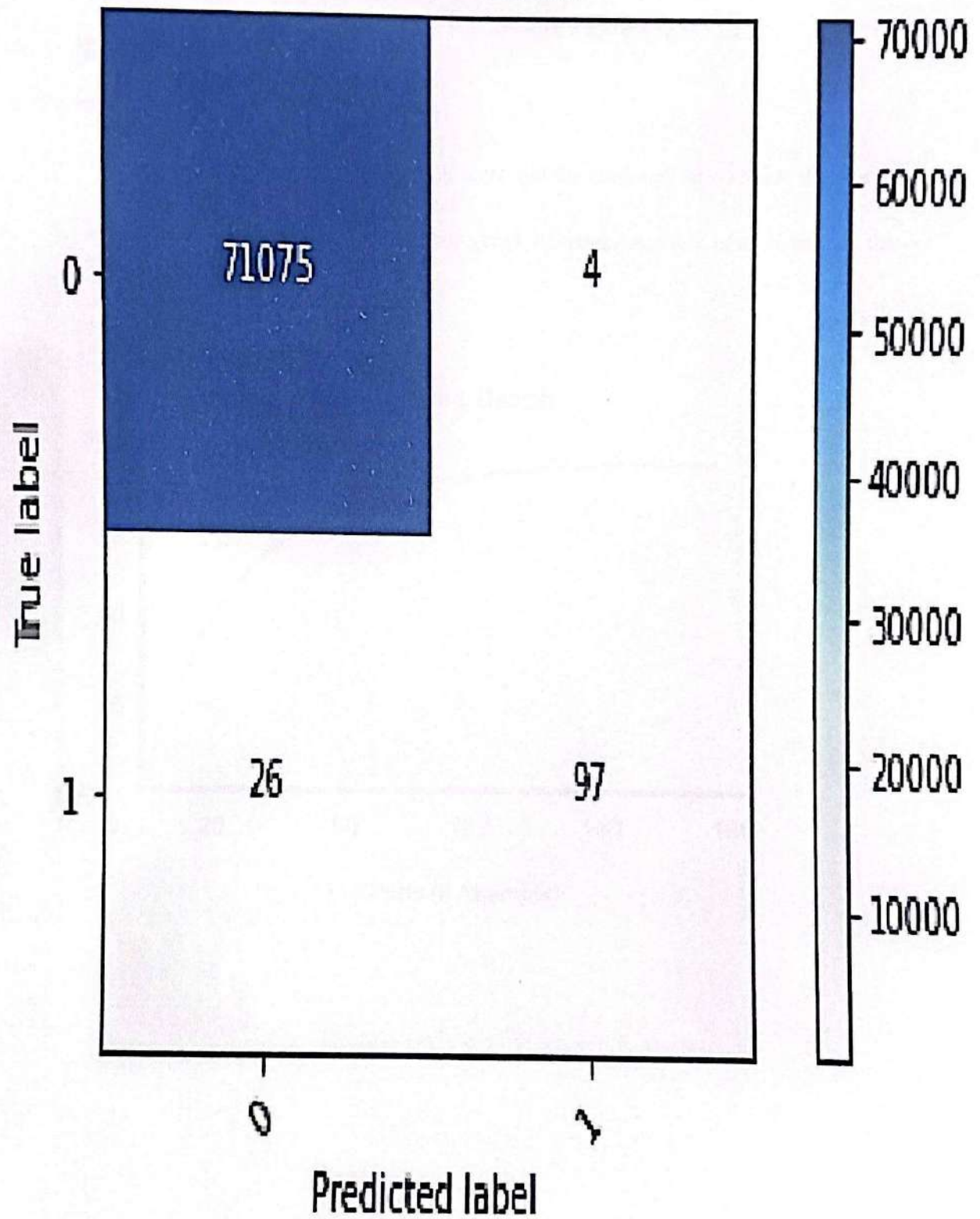
```
    plt.tight_layout()
```

```
plot_confusion_matrix(cnf_matrix, classes=[0,1])
```

```
Confusion matrix, without normalization
```

```
[[71075  4]
```

Confusion matrix



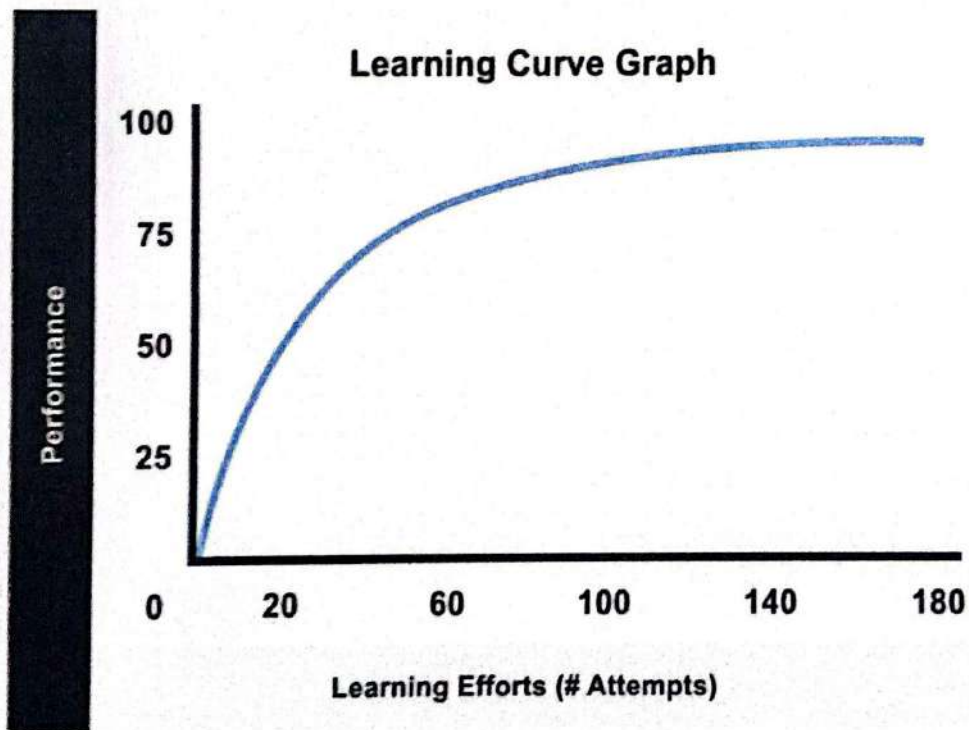
CHAPTER 5

RESULT ANALYSIS

5. Result Analysis

Learning Curves:

- The wider the gap between the training score and the cross-validation score, the more likely your model is overfitting (high variance).
- If the score is low in both training and cross-validation sets this is an indication that our model is underfitting (high bias)



Metrics:

In order to compare various techniques we calculate the true positive, true negative, false positive and false negative generated by a system or an algorithm and use these in quantitative measurements to evaluate and compare performance of different systems.

True Positive (TP) is number of transactions that were fraudulent and were also classified as fraudulent by the system. True Negative (TN) is number of transactions that were legitimate and were also classified as legitimate. False Positive (FP) is number of transactions that were legitimate but were wrongly classified as fraudulent transactions. False Negative (FN) is number of transactions that were fraudulent but were wrongly classified as legitimate transactions by the system.

CHAPTER 6

FUTURE SCOPE

6. Future scope

With increasing number of bank fraudulency and cyber crime cases, need of a secure testing system is on rise. And this is a direct solution to this problem. It can be extended to a duplex verification of not only a customer(debit-ant) but also of the seller(credit-ant). It can be taken and used on a regular basis just like OTP. It can be used to even assess past transaction in database to find whether certain transactions were fraudulent or not and also would be able to produce evidence in such cases. Also optimization techniques on the proposed models and testing on new models can be done.

CONCLUSION

7. Conclusion

Although there are several fraud detection techniques available today, none is able to detect all frauds completely when they are actually happening, they usually detect it after the fraud has been committed. This happens because a very minuscule number of transactions from the total transactions are actually fraudulent in nature. So we need a technology that can detect the fraudulent transaction when it is taking place so that it can be stopped then and there and that too in a minimum cost. So the major task of today is to build an accurate, precise and fast detecting fraud detection system for credit card frauds that can detect not only frauds happening over the internet like phishing and site cloning but also tampering with the credit card itself i.e. it signals an alarm when the tampered credit card is being used. The major drawback of all the techniques is that they are not guaranteed to give the same results in all environments. They give better results with a particular type of dataset and poor or unsatisfactory results with other type. Thus, the results are purely dependent on the dataset type used.

In our undersample data, our model is unable to detect for a large number of cases, the non fraud transactions correctly and instead, mis-classifies those non fraud transactions as fraud cases. Imagine that people that were making regular purchases got their card blocked due to the reason that our model classified that transaction as a fraud transaction, this will be a huge disadvantage for the financial institution. The number of customer complaints and customer dissatisfaction will increase. The next step of this analysis will be to do an outlier removal on our undersample dataset and see if our accuracy in the test set improves.

In this paper, Machine learning technique like Logistic regression, Decision Tree, Random forest were used to detect the fraud in credit card system. Sensitivity, Specificity, accuracy and error rate are used to evaluate the performance for the proposed system.. that Machine learning especially Random and Logistic Regression is the best model for our system.

REFERENCES

1. Y. Sahin, S. Bulkan, and E. Duman, "A cost-sensitive decision tree approach for fraud detection," *Expert Syst. Appl.*, vol. 40, no. 15, pp. 5916–5923, 2013.
2. Sahil Dhankhad, Emad A. Mohammed and Behrouz Far, "Supervised Machine Learning Algorithms for Credit Card Fraudulent Transaction Detection: A Comparative Study "
3. Navanshu Khare and Saad Yunus Sait, "Credit Card Fraud Detection Using Machine Learning Models and Collating Machine Learning Models "
4. Dal Pozzolo, Andrea, et al. "Calibrating Probability with Undersampling for Unbalanced Classification." *Computational Intelligence*, 2015 IEEE Symposium Series on. IEEE, 2015
5. Lakshmi S V S S1, Selvani Deepthi Kavila2 "Machine Learning For Credit Card Fraud Detection System"
6. S. J. K. T. J. C. W. Siddhatha Bhattacharya, "Data Mining for credit card fraud: A comparative study," *Elsevire*, vol. 50, no. 3, pp. 602- 613, 2011.
7. E. D. Y. Sahin, "Detecting credit card fraud by decision trees," in *Proceedings of the international multiconference of engineers and computer science*, Hong Kong, 2011.